Mastering React & TypeScript -- Architecture, Patterns, and Enterprise Development Guide

---

Author: Senior Frontend Architect **Target Audience:** Frontend developers aiming for senior/architect-level React expertise **Prerequisites:** JavaScript, TypeScript fundamentals, HTML/CSS

Table of Contents

1. Introduction to Modern Frontend Development
2. Core Concepts of React
3. Project Architecture and Flow
4. Components and Props
5. React Hooks Deep Dive
6. State Management Principles
7. Context API, Redux, and Zustand
8. React with TypeScript
9. React Routing
10. API Integration
11. Performance Optimization
12. Form Handling
13. Real-World Application Examples
14. React JS vs React TypeScript
15. Production Architecture
16. Common Interview Questions
17. Coding Interview Problems

Part 1: Introduction to Modern Frontend Development

1.1 Evolution of Frontend Development

The Journey from Static Pages to Dynamic Applications

Frontend development has undergone a dramatic transformation over the past three decades. Understanding this evolution is essential to appreciating why modern frameworks like React exist and what problems they solve.

Era 1 -- Static HTML (1991-1999)

The earliest web pages were pure HTML documents. Browsers rendered static content with no interactivity. Every user action required a full page reload from the server.

```
+-------------------+         +-------------------+
|                   |  HTTP   |                   |
|     Browser       | ------> |     Server        |
|  (Renders HTML)   | <------ |  (Sends HTML)     |
|                   |         |                   |
+-------------------+         +-------------------+
     Full page reload on every interaction
```

Era 2 -- Dynamic HTML and JavaScript (1999-2005)

JavaScript enabled client-side interactivity. DHTML, AJAX (Asynchronous JavaScript and XML), and libraries like jQuery allowed partial page updates without full reloads.

```
+-------------------+         +-------------------+
|                   |  AJAX   |                   |
|     Browser       | ------> |     Server        |
|  JS + DOM manip.  | <------ |  (JSON/XML)       |
|                   |         |                   |
+-------------------+         +-------------------+
     Partial updates via XMLHttpRequest
```

Era 3 -- MVC Frameworks (2010-2014)

Frameworks like AngularJS, Backbone.js, and Ember.js introduced structured patterns (MVC/MVVM) to manage growing frontend complexity. Two-way data binding became popular.

Era 4 -- Component-Based Architecture (2013-Present)

React (2013), Vue (2014), and Angular 2+ (2016) shifted the paradigm to component-based architectures. The UI is built from self-contained, reusable building blocks.

```
+-------------------------------------------------------------+
|                    FRONTEND EVOLUTION TIMELINE               |
+-------------------------------------------------------------+
|                                                             |
|  1991        1999         2006        2013        2020+     |
|   |           |            |           |            |        |
|   v           v            v           v            v        |
| Static     Dynamic      jQuery/    Component    Full-Stack   |
|  HTML     HTML + JS      AJAX      Frameworks    JS Apps     |
|                                   (React/Vue)  (Next/Remix)  |
|                                                             |
| Complexity: Low ---------------------------------> Very High |
| Interactivity: None ----------------------------> Rich SPAs |
+-------------------------------------------------------------+
```

Why This Evolution Matters

Each era solved problems of the previous one while introducing new challenges:

Era	Problem Solved	New Challenge
Static HTML	Content delivery	No interactivity
Dynamic JS/AJAX	Partial updates	Spaghetti code, DOM manipulation complexity
MVC Frameworks	Code organization	Performance issues, complex two-way binding
Component Architecture	Reusability, one-way data flow	Learning curve, tooling complexity

---

1.2 Why React Became Popular

React was created at Facebook in 2011 by Jordan Walke and open-sourced in 2013. It became the dominant frontend library for several compelling reasons.

Key Reasons for React's Dominance

1. Virtual DOM for Performance

React introduced the Virtual DOM -- a lightweight JavaScript representation of the actual DOM. Instead of manipulating the real DOM directly (which is slow), React computes the minimal set of changes needed and applies them efficiently.

2. Component-Based Architecture

React broke the UI into small, reusable, isolated components. Each component manages its own state and rendering logic, making large applications maintainable.

3. Declarative Programming Model

Developers describe WHAT the UI should look like for a given state, not HOW to update it. React handles the DOM updates behind the scenes.

```
// Imperative (jQuery style) -- HOW to update
document.getElementById('counter').innerHTML = count;

// Declarative (React style) -- WHAT to render
<span>{count}</span>
```

4. One-Way Data Flow (Unidirectional)

Data flows downward from parent to child via props, making the application state predictable and easier to debug.

```
+-------------------+
|   Parent (state)  |
+--------+----------+
         | props
         v
+--------+----------+
|   Child (render)  |
+-------------------+
```

5. Rich Ecosystem

React is a library, not a full framework. This means developers can choose their own routing (React Router), state management (Redux, Zustand), and other tools, creating a flexible ecosystem.

6. Strong Community and Corporate Backing

Backed by Meta (Facebook) and used by companies like Netflix, Airbnb, Instagram, and Uber, React has massive community support, extensive documentation, and thousands of third-party packages.

1.3 Problems React Solves

The Core Problems

Problem	How React Solves It
Slow DOM manipulation	Virtual DOM with efficient diffing algorithm
Code duplication	Reusable components
Complex UI state management	Declarative rendering + hooks
Unpredictable data flow	Unidirectional data flow
Tight coupling of concerns	Component encapsulation
Manual DOM synchronization	Automatic re-rendering on state change

Before React vs After React

```
BEFORE REACT (Manual DOM Management):
+-------------------------------------------------------------------+
|  HTML Template  -->  JS Logic  -->  Manual DOM Updates            |
|       |                  |                  |                     |
|       +------------------+-------------------+                    |
|       Tightly coupled, fragile, hard to maintain                  |
+-------------------------------------------------------------------+

AFTER REACT (Component-Based):
+-------------------------------------------------------------------+
|   Component A          Component B          Component C           |
|   +-----------+        +-----------+        +-----------+         |
|   | State     |        | State     |        | State     |         |
|   | Logic     |        | Logic     |        | Logic     |         |
|   | Render    |        | Render    |        | Render    |         |
|   +-----------+        +-----------+        +-----------+         |
|        |                    |                    |                 |
|        +--------------------+--------------------+                |
|        Isolated, reusable, independently testable                 |
+-------------------------------------------------------------------+
```

1.4 SPA vs Traditional Web Applications

Traditional Multi-Page Application (MPA)

In a traditional MPA, each user interaction triggers a full HTTP request to the server, which returns a complete new HTML page.

```
+--------+    Request /home     +---------+
| Browser| ------------------> |  Server  |
|        | <------------------ |          |
|        |   Full HTML page    |          |
|        |                     |          |
|        |    Request /about   |          |
|        | ------------------> |          |
|        | <------------------ |          |
|        |   Full HTML page    |          |
+--------+                     +---------+
     Every navigation = full page reload
```

Single Page Application (SPA)

In an SPA, the browser loads a single HTML page initially. Subsequent "navigations" are handled entirely on the client side by JavaScript -- only data (JSON) is fetched from the server.

```
+--------+   Initial Request   +---------+
| Browser| ------------------> |  Server  |
|        | <------------------ |          |
|        |  HTML + JS Bundle   |          |
|        |                     |          |
|        |   API Call (JSON)   |          |
|        | ------------------> |          |
|        | <------------------ |          |
|        |   { data: ... }     |          |
+--------+                     +---------+
     Navigation handled by JS (React Router)
     Only data is fetched, not full pages
```

Comparison Table

Feature	Traditional MPA	Single Page Application (SPA)
Page Load	Full reload on each navigation	Initial load only, then dynamic updates

Speed	Slower (full HTML each time)	Faster after initial load
User Experience	Page flickers on navigation	Smooth, app-like experience
SEO	Naturally good (server-rendered HTML)	Requires SSR/SSG for SEO
Server Load	Higher (renders HTML each time)	Lower (serves JSON APIs)
Complexity	Lower	Higher (client-side routing, state)
Offline Support	None	Possible with service workers
Example Frameworks	Rails, Django, Spring MVC	React, Angular, Vue

When to Choose Each

Choose MPA when:

- SEO is critical and you cannot use SSR
- The application is content-heavy with minimal interactivity
- The team is more comfortable with server-side rendering

Choose SPA when:

- Rich interactivity is required (dashboards, editors, real-time apps)
- App-like user experience is important
- The API backend is already built or will serve multiple clients

> ***Note:*** *Modern frameworks like Next.js and Remix blur the line between MPA and SPA by offering Server-Side Rendering (SSR), Static Site Generation (SSG), and client-side navigation in the same application.*

SPA Architecture with React

```
+-----------------------------------------------------------------+
|                        SPA Architecture                          |
+-----------------------------------------------------------------+
|                                                                  |
|  +-------------------------------------------------------------+ |
|  |                     React Application                        | |
|  |                                                              | |
|  |  +----------+  +----------+  +----------+  +----------+    | |
|  |  |  Header  |  |  Sidebar |  |  Content |  |  Footer  |    | |
|  |  +----------+  +----------+  +----+-----+  +----------+    | |
|  |                                   |                         | |
```

```
|  |                          +--------+--------+               | |
|  |                          |                 |               | |
|  |                     +----+----+       +----+----+          | |
|  |                     | Page A  |       | Page B  |          | |
|  |                     +---------+       +---------+          | |
|  |                                                            | |
|  +------------------------------------------------------------+ |
|                          |                                       |
|                    React Router                                  |
|                (Client-Side Routing)                              |
|                          |                                       |
|  +------------------------------------------------------------+ |
|  |                   HTTP / REST / GraphQL                     | |
|  +------------------------------------------------------------+ |
|                          |                                       |
|  +------------------------------------------------------------+ |
|  |                     Backend API Server                       | |
|  |              (Node.js / Spring Boot / etc.)                  | |
|  +------------------------------------------------------------+ |
+-------------------------------------------------------------------+
```

Part 2: Core Concepts of React

2.1 Virtual DOM

What Is the Virtual DOM?

The Virtual DOM (VDOM) is a lightweight, in-memory JavaScript representation of the real DOM. It is a programming concept implemented by React where a virtual copy of the UI is kept in memory and synced with the real DOM through a process called **reconciliation**.

Why Does the Virtual DOM Exist?

Direct DOM manipulation is expensive. Every time you change a DOM element, the browser must:

1. Re-calculate CSS styles (Recalculate Style)
2. Re-compute layout (Reflow)
3. Re-paint pixels on the screen (Repaint)

These operations are costly, especially in complex UIs with frequent updates. The Virtual DOM minimizes these operations by batching changes and computing the minimal set of updates.

How the Virtual DOM Works

```
STEP 1: Render Virtual DOM
+-----------------------------------+
|         Virtual DOM (v1)          |
|  +------+  +------+  +------+   |
|  | div  |  |  h1  |  |  p   |   |
|  +------+  +------+  +------+   |
+-----------------------------------+

STEP 2: State Changes -> New Virtual DOM
+-----------------------------------+
|         Virtual DOM (v2)          |
|  +------+  +------+  +------+   |
|  | div  |  |  h1  |  | span |   | <-- p changed to span
|  +------+  +------+  +------+   |
+-----------------------------------+

STEP 3: Diffing (comparing v1 and v2)
+-----------------------------------+
|          Diff Result              |
|  div  -> No change               |
```

```
|  h1   -> No change              |
|  p    -> CHANGED to span        |
+-----------------------------------+

STEP 4: Patch Real DOM (only the changed part)
+-----------------------------------+
|           Real DOM                |
|  div  -> unchanged              |
|  h1   -> unchanged              |
|  span -> UPDATED (was p)        |
+-----------------------------------+
```

Virtual DOM vs Real DOM

Feature	Real DOM	Virtual DOM
Nature	Actual browser DOM tree	JavaScript object representation
Update Speed	Slow (triggers reflow/repaint)	Fast (in-memory operations)
Updates	Updates entire tree or subtree	Computes minimal diff
Memory	Browser-managed	Application-managed (JS heap)
Direct Manipulation	Yes	No (updates go through React)

Code Example: What Virtual DOM Looks Like Internally

When you write JSX:

```
const element = <h1 className="title">Hello, React!</h1>;
```

React converts this to a Virtual DOM object:

```
// What React creates internally
const vdomElement = {
  type: 'h1',
  props: {
    className: 'title',
    children: 'Hello, React!'
  }
};
```

This lightweight object is much cheaper to create and compare than actual DOM nodes.

2.2 Component Architecture

What Is Component Architecture?

In React, the entire UI is built from **components** -- self-contained, reusable pieces of UI that manage their own state and rendering. Each component is responsible for a specific part of the interface.

Why Component Architecture?

Problem	Solution via Components
Monolithic HTML files	Small, focused components
Code duplication	Reusable components across pages
Difficult testing	Each component can be tested in isolation
Team collaboration issues	Different developers work on different components
Maintenance nightmare	Changes are localized to individual components

Component Tree

Every React application has a tree structure of components. The root component (usually `App`) contains child components, which can contain their own children.

```
                      App
                       |
         +--------------+--------------+
         |              |              |
       Header         Main          Footer
         |              |
    +----+----+    +----+----+
    |         |    |         |
  Logo     NavBar  Sidebar  Content
                            |
                  +---------+---------+
                  |                   |
               ArticleList        SearchBar
                  |
            +-----+-----+
```

```
              |            |
         ArticleCard  ArticleCard
```

Component Types

1. Presentational Components (UI Components)

Components focused purely on how things look. They receive data via props and render UI.

```
// Presentational -- only renders UI
function UserAvatar({ name, imageUrl }) {
  return (
    <div className="avatar">
      <img src={imageUrl} alt={name} />
      <span>{name}</span>
    </div>
  );
}
```

2. Container Components (Smart Components)

Components that manage state, handle logic, and pass data to presentational components.

```
// Container -- manages data and logic
function UserProfile() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetchUser().then(setUser);
  }, []);

  if (!user) return <Loading />;
  return <UserAvatar name={user.name} imageUrl={user.avatar} />;
}
```

3. Layout Components

Components that define the page structure.

```
function DashboardLayout({ children }) {
  return (
```

```
    <div className="dashboard">
      <Sidebar />
      <main className="content">{children}</main>
    </div>
  );
}
```

Real-World Component Architecture

```
+-------------------------------------------------------------------+
|                    E-Commerce Application                         |
+-------------------------------------------------------------------+
|                                                                   |
|  Shared Components        Feature Components    Page Components  |
|  +----------------+       +----------------+   +---------------+ |
|  | Button         |       | ProductCard    |   | HomePage      | |
|  | Input          |       | CartItem       |   | ProductPage   | |
|  | Modal          |       | CheckoutForm   |   | CartPage      | |
|  | Spinner        |       | ReviewList     |   | CheckoutPage  | |
|  | Card           |       | CategoryFilter |   | ProfilePage   | |
|  +----------------+       +----------------+   +---------------+ |
|                                                                   |
|  Layout Components         HOC / Wrappers                         |
|  +----------------+       +----------------+                     |
|  | MainLayout     |       | AuthGuard      |                     |
|  | DashboardLayout|       | ErrorBoundary  |                     |
|  | AuthLayout     |       | ThemeProvider   |                     |
|  +----------------+       +----------------+                     |
+-------------------------------------------------------------------+
```

2.3 Declarative UI

What Is Declarative UI?

Declarative UI means describing **what** the UI should look like for a given state, rather than specifying **how** to manipulate the DOM to achieve that result. React automatically handles the DOM updates when the state changes.

Imperative vs Declarative Approach

Imperative Approach (Vanilla JavaScript):

```
// IMPERATIVE: Step-by-step instructions for HOW to update
const list = document.getElementById('todo-list');

function addItem(text) {
  const li = document.createElement('li');
  li.textContent = text;
  li.addEventListener('click', function() {
    li.classList.toggle('completed');
  });
  list.appendChild(li);
}

function removeItem(index) {
  const items = list.querySelectorAll('li');
  if (items[index]) {
    list.removeChild(items[index]);
  }
}
```

Declarative Approach (React):

```
// DECLARATIVE: Describe WHAT the UI should look like
function TodoList() {
  const [items, setItems] = useState([]);

  return (
    <ul>
      {items.map((item, index) => (
        <li
          key={index}
          className={item.completed ? 'completed' : ''}
          onClick={() => toggleItem(index)}
        >
          {item.text}
        </li>
      ))}
    </ul>
  );
}
```

Why Declarative Is Better

Aspect	Imperative	Declarative
Focus	HOW (step-by-step DOM manipulation)	WHAT (desired UI state)
Code Readability	Harder to follow the logic	Reads like a description of the UI
Bug Proneness	High (manual sync between state and DOM)	Low (React handles synchronization)
Testability	Difficult (DOM-dependent)	Easier (test state -> render output)
Maintenance	Hard (fragile DOM references)	Easy (change state, UI updates automatically)

The Declarative Mental Model

```
+-----------+    triggers    +---------------+    produces    +--------+
|   Event   | ------------> |  State Change | ------------> |  New   |
| (click,   |               | (setState)    |               |  UI    |
|  input,   |               |               |               | (JSX)  |
|  fetch)   |               |               |               |        |
+-----------+               +---------------+               +--------+
                                                                |
                                                                v
                                                        React handles
                                                        DOM updates
                                                        automatically
```

> ***Key Insight:*** *In declarative UI, you never touch the DOM directly. You update the state, and React takes care of reflecting that state in the UI.*

2.4 Reconciliation Process

What Is Reconciliation?

Reconciliation is React's algorithm for determining what changed between two renders and updating the DOM efficiently. When a component's state or props change, React creates a new Virtual DOM tree and compares it with the previous one. This comparison process is called **diffing**, and the overall process of updating the real DOM based on the diff is **reconciliation**.

Why Reconciliation Is Necessary

Without reconciliation, React would have to re-render the entire DOM tree on every state change, which would be extremely slow. Reconciliation ensures that only the minimal set of changes are applied to the real DOM.

How Reconciliation Works -- The Diffing Algorithm

React's diffing algorithm operates with two key assumptions (heuristics) that make it efficient (O(n) complexity instead of O(n^3)):

Heuristic 1: Different element types produce different trees

If the root elements of two trees are different types, React tears down the old tree and builds the new one from scratch.

```
// Old render                // New render
<div>                        <span>
  <Counter />                  <Counter />
</div>                       </span>

// React destroys the entire <div> subtree
// and builds a new <span> subtree from scratch
```

Heuristic 2: Keys identify stable elements in lists

Developer-supplied `key` props tell React which items in a list are the same across renders.

```
// Without keys -- React re-renders all items
<ul>
  <li>Apple</li>
  <li>Banana</li>    {/* Adding "Mango" at top re-renders everything */}
</ul>

// With keys -- React knows which items are stable
<ul>
  <li key="apple">Apple</li>
  <li key="banana">Banana</li>  {/* Adding "Mango" only adds one node */}
</ul>
```

Reconciliation Flow

```
+-------------------------------------------------------------+
|                    RECONCILIATION PROCESS                     |
+-------------------------------------------------------------+
|                                                              |
|  1. State/Props Change                                       |
|     |                                                        |
|     v                                                        |
|  2. Component re-renders -> New Virtual DOM tree             |
|     |                                                        |
|     v                                                        |
|  3. DIFFING: Compare new VDOM with previous VDOM            |
|     |                                                        |
|     +---> Same type? --> Compare attributes/props            |
|     |         |           Update only changed attributes     |
|     |         v                                              |
|     |     Recurse on children                                |
|     |                                                        |
|     +---> Different type? --> Destroy old subtree            |
|               |                Build new subtree             |
|               v                                              |
|           Mount new components                               |
|                                                              |
|  4. Compute minimal set of DOM operations                    |
|     |                                                        |
|     v                                                        |
|  5. COMMIT: Apply changes to Real DOM in a batch             |
|     |                                                        |
|     v                                                        |
|  6. Browser paints updated UI                                |
+-------------------------------------------------------------+
```

React Fiber Architecture

Starting with React 16, the reconciliation algorithm was rewritten as **React Fiber**. Fiber introduced:

Feature	Description
Incremental rendering	Breaking rendering work into chunks
Priority-based updates	Urgent updates (user input) processed before non-urgent (data fetch)
Pause and resume	Ability to pause rendering to handle higher-priority work
Abort	Ability to abort in-progress rendering if it becomes unnecessary
Concurrency	Work on multiple state updates simultaneously

```
+-------------------------------------------------------------+
|                    REACT FIBER ARCHITECTURE                  |
+-------------------------------------------------------------+
|                                                             |
|  +------------------+                                       |
|  | Component Update | --+                                  |
|  +------------------+   |                                   |
|                         v                                   |
|  +------------------+   +------------------+                |
|  | Another Update   |-->| FIBER SCHEDULER  |                |
|  +------------------+   |                  |                |
|                         | Prioritizes work |                |
|  +------------------+   | into lanes       |                |
|  | User Interaction |-->|                  |                |
|  +------------------+   +--------+---------+                |
|                                  |                          |
|                   +-------------+-------------+             |
|                   |                           |             |
|             +-----+------+          +--------+-------+      |
|             | High Prio  |          |  Low Priority  |      |
|             | (Sync Lane)|          |  (Transition)  |      |
|             | User input |          |  Data fetch    |      |
|             +-----+------+          +--------+-------+      |
|                   |                           |             |
|                   v                           v             |
|             Process                    Process when         |
|             immediately                idle / interruptible |
+-------------------------------------------------------------+
```

Practical Implications of Reconciliation

1. Always use unique, stable keys for lists:

```
// GOOD: Stable, unique key
{users.map(user => (
  <UserCard key={user.id} user={user} />
))}

// BAD: Index as key (problematic when list order changes)
{users.map((user, index) => (
  <UserCard key={index} user={user} />
))}
```

2. Keep component structure stable:

```
// BAD: Conditional changes element type, causes full remount
{isAdmin ? <AdminPanel /> : <div><UserPanel /></div>}

// GOOD: Keep structure consistent
{isAdmin ? <AdminPanel /> : <UserPanel />}
```

3. Lift state up to prevent unnecessary re-renders:

When multiple sibling components depend on the same state, lift that state to the nearest common ancestor to ensure efficient reconciliation.

Summary of Core Concepts

```
+-------------------------------------------------------------+
|              REACT CORE CONCEPTS SUMMARY                     |
+-------------------------------------------------------------+
|                                                             |
|  Virtual DOM         In-memory representation of real DOM   |
|                      Enables efficient updates              |
|                                                             |
|  Components          Self-contained UI building blocks      |
|                      Manage own state and rendering         |
|                                                             |
|  Declarative UI      Describe WHAT, not HOW                 |
|                      State drives the UI automatically      |
|                                                             |
|  Reconciliation      Diffing algorithm for minimal updates  |
|                      Fiber architecture for concurrent work |
|                                                             |
|  One-Way Data Flow   Props flow down, events flow up        |
|                      Predictable and debuggable             |
+-------------------------------------------------------------+
```

Part 3: React Project Architecture

3.1 Why Project Architecture Matters

A well-organized project structure is critical for production-grade React applications. Without a clear architecture, codebases become tangled, onboarding new developers becomes painful, and maintaining the application becomes exponentially harder as it grows.

Goals of Good Architecture

Goal	Description
Scalability	Easy to add new features without restructuring
Maintainability	Easy to find, understand, and modify code
Reusability	Components and utilities shared across features
Testability	Each layer can be tested independently
Team Collaboration	Multiple developers can work without conflicts

---

3.2 Recommended Project Structure

Production-Grade Folder Structure

```
src/
|-- assets/              # Static files: images, fonts, icons
|   |-- images/
|   |-- fonts/
|   +-- icons/
|
|-- components/           # Shared/reusable UI components
|   |-- Button/
|   |   |-- Button.tsx
|   |   |-- Button.styles.css
|   |   +-- Button.test.tsx
|   |-- Modal/
|   |-- Input/
|   +-- Card/
|
```

```
|-- pages/                # Page-level components (route targets)
|   |-- HomePage/
|   |   |-- HomePage.tsx
|   |   +-- HomePage.styles.css
|   |-- DashboardPage/
|   |-- LoginPage/
|   +-- ProfilePage/
|
|-- hooks/                # Custom React hooks
|   |-- useAuth.ts
|   |-- useFetch.ts
|   |-- useDebounce.ts
|   +-- useLocalStorage.ts
|
|-- services/             # API calls and external service integrations
|   |-- api.ts            # Axios/fetch instance configuration
|   |-- authService.ts
|   |-- userService.ts
|   +-- productService.ts
|
|-- context/              # React Context providers
|   |-- AuthContext.tsx
|   |-- ThemeContext.tsx
|   +-- CartContext.tsx
|
|-- utils/                # Pure utility functions
|   |-- formatDate.ts
|   |-- validators.ts
|   |-- constants.ts
|   +-- helpers.ts
|
|-- types/                # TypeScript type definitions
|   |-- user.ts
|   |-- product.ts
|   +-- api.ts
|
|-- layouts/              # Layout wrapper components
|   |-- MainLayout.tsx
|   |-- AuthLayout.tsx
|   +-- DashboardLayout.tsx
|
|-- routes/               # Route configuration
|   |-- AppRoutes.tsx
|   +-- ProtectedRoute.tsx
|
|-- store/                # State management (Redux/Zustand)
|   |-- slices/
|   |-- store.ts
```

```
|   +-- hooks.ts
|
|-- App.tsx                # Root application component
|-- index.tsx              # Entry point
+-- index.css              # Global styles
```

Why Each Folder Exists

Folder	Purpose	Example Contents
`components/`	Shared, reusable UI building blocks	Button, Modal, Card, Input
`pages/`	Top-level views mapped to routes	HomePage, LoginPage, Dashboard
`hooks/`	Custom hooks encapsulating reusable logic	useAuth, useFetch, useDebounce
`services/`	API communication layer	authService, userService
`context/`	React Context for cross-cutting state	AuthContext, ThemeContext
`utils/`	Pure helper functions (no React dependency)	formatDate, validators
`types/`	TypeScript interfaces and type definitions	User, Product, ApiResponse
`layouts/`	Page layout wrappers (header, sidebar, footer)	MainLayout, DashboardLayout
`routes/`	Route definitions and guards	AppRoutes, ProtectedRoute
`store/`	Global state management	Redux slices, Zustand stores
`assets/`	Static resources	Images, fonts, SVG icons

3.3 Architecture Diagram

Layered Architecture

```
+-----------------------------------------------------------------+
|                    REACT APPLICATION LAYERS                      |
+-----------------------------------------------------------------+
|                                                                  |
|   PRESENTATION LAYER                                             |
|   +-------------------------------------------------------------+ |
|   |  Pages          Components       Layouts                    | |
|   |  (Route Views) (Reusable UI)    (Page Wrappers)             | |
|   +-------------------------------------------------------------+ |
```

```
|                          |                                          |
|   LOGIC LAYER                                                       |
|   +-------------------------------------------------------------+ |
|   |  Hooks          Context         Store (Redux/Zustand)      | |
|   |  (Custom Logic) (Shared State)  (Global State)             | |
|   +-------------------------------------------------------------+ |
|                          |                                          |
|   DATA LAYER                                                        |
|   +-------------------------------------------------------------+ |
|   |  Services        Utils           Types                     | |
|   |  (API Calls)     (Helpers)       (TypeScript Definitions)  | |
|   +-------------------------------------------------------------+ |
|                          |                                          |
|   EXTERNAL                                                          |
|   +-------------------------------------------------------------+ |
|   |  REST APIs       GraphQL        Third-Party Services       | |
|   +-------------------------------------------------------------+ |
+-----------------------------------------------------------------+
```

Data Flow Architecture

```
+-----------------------------------------------------------------+
|                    DATA FLOW IN REACT APP                        |
+-----------------------------------------------------------------+
|                                                                  |
|  User Event (click, type, submit)                                |
|       |                                                          |
|       v                                                          |
|  Component Handler (onClick, onChange)                           |
|       |                                                          |
|       +--> Local State? --> useState/useReducer --> Re-render    |
|       |                                                          |
|       +--> Global State? --> dispatch(action) --> Store Update   |
|       |                          |                               |
|       |                          v                               |
|       |                  All subscribed components re-render     |
|       |                                                          |
|       +--> API Call? --> Service Layer --> fetch/axios           |
|                              |                                   |
|                              v                                   |
|                        Backend API                               |
|                              |                                   |
|                              v                                   |
```

```
|                        Response -> Update State -> Re-render     |
+-------------------------------------------------------------------+
```

Feature-Based Alternative Structure

For larger applications, some teams prefer organizing by feature instead of by type:

```
src/
|-- features/
|   |-- auth/
|   |   |-- components/
|   |   |   |-- LoginForm.tsx
|   |   |   +-- RegisterForm.tsx
|   |   |-- hooks/
|   |   |   +-- useAuth.ts
|   |   |-- services/
|   |   |   +-- authService.ts
|   |   |-- context/
|   |   |   +-- AuthContext.tsx
|   |   |-- pages/
|   |   |   |-- LoginPage.tsx
|   |   |   +-- RegisterPage.tsx
|   |   +-- types.ts
|   |
|   |-- products/
|   |   |-- components/
|   |   |-- hooks/
|   |   |-- services/
|   |   +-- pages/
|   |
|   +-- cart/
|       |-- components/
|       |-- hooks/
|       +-- services/
|
|-- shared/                # Shared across all features
|   |-- components/
|   |-- hooks/
|   |-- utils/
|   +-- types/
|
|-- App.tsx
+-- index.tsx
```

Structure	Best For	Trade-off

Type-based (components/, hooks/, etc.)	Small to medium apps, teams new to React	Harder to find all files for a feature
Feature-based (features/auth/, etc.)	Large apps, multiple teams	Some duplication, more folders

---

Part 4: React Components

4.1 Functional Components

What Are Functional Components?

Functional components are JavaScript functions that accept `props` as an argument and return JSX (React elements). They are the modern standard for writing React components.

Why Use Functional Components?

- Simpler syntax -- just a function
- Hooks enable full state and lifecycle capabilities
- Easier to test and reason about
- Better performance (no class instantiation overhead)
- The recommended approach by the React team since React 16.8

JavaScript Example

```
// JavaScript Functional Component
import React from 'react';

function Greeting({ name, age }) {
  return (
    <div className="greeting">
      <h2>Hello, {name}!</h2>
      <p>You are {age} years old.</p>
    </div>
  );
}

// Arrow function variant
const Greeting = ({ name, age }) => (
  <div className="greeting">
    <h2>Hello, {name}!</h2>
    <p>You are {age} years old.</p>
  </div>
);

// Usage
<Greeting name="Alice" age={28} />
```

TypeScript Example

```
// TypeScript Functional Component
import React from 'react';

// Define props interface
interface GreetingProps {
  name: string;
  age: number;
  isActive?: boolean; // optional prop
}

const Greeting: React.FC<GreetingProps> = ({ name, age, isActive = true }) => {
  return (
    <div className={`greeting ${isActive ? 'active' : ''}`}>
      <h2>Hello, {name}!</h2>
      <p>You are {age} years old.</p>
    </div>
  );
};

// Alternative: Typing props directly (preferred by many teams)
function Greeting({ name, age, isActive = true }: GreetingProps) {
  return (
    <div className={`greeting ${isActive ? 'active' : ''}`}>
      <h2>Hello, {name}!</h2>
      <p>You are {age} years old.</p>
    </div>
  );
}

// Usage
<Greeting name="Alice" age={28} />
<Greeting name="Bob" age={32} isActive={false} />
```

4.2 Class Components

What Are Class Components?

Class components are ES6 classes that extend `React.Component`. They were the primary way to create stateful components before React 16.8 introduced hooks. While still supported, they are considered legacy for new code.

When to Use Class Components

- Maintaining legacy codebases that already use them
- Using `Error Boundaries` (which still require class components as of React 18)
- Understanding them for interviews and older code

JavaScript Example

```
// JavaScript Class Component
import React, { Component } from 'react';

class UserProfile extends Component {
  constructor(props) {
    super(props);
    this.state = {
      isEditing: false,
      bio: props.initialBio || ''
    };
  }

  toggleEdit = () => {
    this.setState(prevState => ({ isEditing: !prevState.isEditing }));
  };

  handleBioChange = (e) => {
    this.setState({ bio: e.target.value });
  };

  render() {
    const { name, email } = this.props;
    const { isEditing, bio } = this.state;

    return (
      <div className="profile">
        <h2>{name}</h2>
        <p>{email}</p>
        {isEditing ? (
          <textarea value={bio} onChange={this.handleBioChange} />
        ) : (
          <p>{bio}</p>
        )}
        <button onClick={this.toggleEdit}>
          {isEditing ? 'Save' : 'Edit'}
        </button>
      </div>
    );
```

```
  }
}
```

TypeScript Example

```
// TypeScript Class Component
import React, { Component } from 'react';

interface UserProfileProps {
  name: string;
  email: string;
  initialBio?: string;
}

interface UserProfileState {
  isEditing: boolean;
  bio: string;
}

class UserProfile extends Component<UserProfileProps, UserProfileState> {
  constructor(props: UserProfileProps) {
    super(props);
    this.state = {
      isEditing: false,
      bio: props.initialBio || ''
    };
  }

  toggleEdit = (): void => {
    this.setState(prevState => ({ isEditing: !prevState.isEditing }));
  };

  handleBioChange = (e: React.ChangeEvent<HTMLTextAreaElement>): void => {
    this.setState({ bio: e.target.value });
  };

  render(): React.ReactNode {
    const { name, email } = this.props;
    const { isEditing, bio } = this.state;

    return (
      <div className="profile">
        <h2>{name}</h2>
        <p>{email}</p>
        {isEditing ? (
```

```
          <textarea value={bio} onChange={this.handleBioChange} />
        ) : (
          <p>{bio}</p>
        )}
        <button onClick={this.toggleEdit}>
          {isEditing ? 'Save' : 'Edit'}
        </button>
      </div>
    );
  }
}
```

4.3 Functional vs Class Components -- Comparison

Feature	Functional Component	Class Component
Syntax	Plain function	ES6 class extending React.Component
State Management	`useState` hook	`this.state` + `this.setState()`
Side Effects	`useEffect` hook	Lifecycle methods (componentDidMount, etc.)
Performance	Slightly better (no class overhead)	Slightly heavier
`this` keyword	Not needed	Required (binding issues)
Error Boundaries	Not supported directly	Supported via componentDidCatch
Code Length	Shorter, more concise	Longer, more boilerplate
Testing	Easier (pure function)	Harder (class instance)
React Team Recommendation	Preferred for new code	Legacy, still supported

---

4.4 Props (Properties)

What Are Props?

Props are read-only data passed from a parent component to a child component. They are the primary mechanism for component communication in React's unidirectional data flow.

Key Characteristics of Props

- **Read-only** -- A child component must never modify its own props
- **Passed from parent** -- Data flows downward
- **Can be any type** -- Strings, numbers, objects, arrays, functions, even other components
- **Trigger re-renders** -- When props change, the child re-renders

Props Flow Diagram

```
+---------------------+
|   ParentComponent   |
|                     |
|  state: {           |
|    user: "Alice",   |
|    role: "Admin"    |
|  }                  |
+--------+------------+
         |
    props: { name="Alice", role="Admin", onLogout={fn} }
         |
         v
+--------+------------+
|   ChildComponent    |
|                     |
|  Receives:          |
|    props.name       |
|    props.role       |
|    props.onLogout   |
+---------------------+
```

JavaScript Example

```javascript
// Parent Component
function App() {
  const [user, setUser] = useState({ name: 'Alice', role: 'admin' });

  const handleLogout = () => {
    setUser(null);
  };

  return (
    <div>
      <UserCard
        name={user.name}
```

```
        role={user.role}
        onLogout={handleLogout}
      />
    </div>
  );
}

// Child Component
function UserCard({ name, role, onLogout }) {
  return (
    <div className="user-card">
      <h3>{name}</h3>
      <span className="badge">{role}</span>
      <button onClick={onLogout}>Logout</button>
    </div>
  );
}
```

TypeScript Example

```
// Define types for props
interface UserCardProps {
  name: string;
  role: 'admin' | 'editor' | 'viewer';
  onLogout: () => void;
  avatar?: string; // optional prop
}

// Child Component with typed props
function UserCard({ name, role, onLogout, avatar }: UserCardProps) {
  return (
    <div className="user-card">
      {avatar && <img src={avatar} alt={name} />}
      <h3>{name}</h3>
      <span className="badge">{role}</span>
      <button onClick={onLogout}>Logout</button>
    </div>
  );
}
```

Children Props

React components can receive child elements through the special `children` prop:

```
// TypeScript
interface CardProps {
  title: string;
  children: React.ReactNode;
}

function Card({ title, children }: CardProps) {
  return (
    <div className="card">
      <div className="card-header">
        <h3>{title}</h3>
      </div>
      <div className="card-body">
        {children}
      </div>
    </div>
  );
}

// Usage
<Card title="User Info">
  <p>Name: Alice</p>
  <p>Email: alice@example.com</p>
</Card>
```

Default Props

```
interface ButtonProps {
  label: string;
  variant?: 'primary' | 'secondary' | 'danger';
  size?: 'sm' | 'md' | 'lg';
  disabled?: boolean;
}

function Button({
  label,
  variant = 'primary',
  size = 'md',
  disabled = false
}: ButtonProps) {
  return (
    <button
      className={`btn btn-${variant} btn-${size}`}
```

```
      disabled={disabled}
    >
      {label}
    </button>
  );
}

// Usage -- defaults applied automatically
<Button label="Submit" />
<Button label="Cancel" variant="secondary" size="sm" />
```

4.5 State

What Is State?

State is mutable data managed within a component that determines the component's behavior and rendering. When state changes, React re-renders the component to reflect the new state.

Props vs State

Feature	Props	State
Owned by	Parent component	The component itself
Mutable?	No (read-only)	Yes (via setState / useState)
Passed down?	Yes, parent to child	No, local to component
Triggers re-render?	Yes, when changed by parent	Yes, when updated
Purpose	Configure a component	Track dynamic data

State in Functional Components (useState)

JavaScript:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  const [step, setStep] = useState(1);

  const increment = () => setCount(prev => prev + step);
  const decrement = () => setCount(prev => prev - step);
```

```
  const reset = () => setCount(0);

  return (
    <div className="counter">
      <h2>Count: {count}</h2>
      <div className="controls">
        <button onClick={decrement}>-</button>
        <button onClick={increment}>+</button>
        <button onClick={reset}>Reset</button>
      </div>
      <label>
        Step:
        <input
          type="number"
          value={step}
          onChange={(e) => setStep(Number(e.target.value))}
        />
      </label>
    </div>
  );
}
```

TypeScript:

```
import React, { useState } from 'react';

interface CounterState {
  count: number;
  step: number;
}

function Counter(): React.ReactElement {
  const [count, setCount] = useState<number>(0);
  const [step, setStep] = useState<number>(1);

  const increment = (): void => setCount(prev => prev + step);
  const decrement = (): void => setCount(prev => prev - step);
  const reset = (): void => setCount(0);

  return (
    <div className="counter">
      <h2>Count: {count}</h2>
      <div className="controls">
        <button onClick={decrement}>-</button>
        <button onClick={increment}>+</button>
        <button onClick={reset}>Reset</button>
```

```
      </div>
      <label>
        Step:
        <input
          type="number"
          value={step}
          onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
            setStep(Number(e.target.value))
          }
        />
      </label>
    </div>
  );
}
```

State with Objects

```
interface User {
  name: string;
  email: string;
  age: number;
}

function UserForm() {
  const [user, setUser] = useState<User>({
    name: '',
    email: '',
    age: 0
  });

  // IMPORTANT: Always create a new object, never mutate state directly
  const updateField = (field: keyof User, value: string | number): void => {
    setUser(prev => ({
      ...prev,
      [field]: value
    }));
  };

  return (
    <form>
      <input
        value={user.name}
        onChange={(e) => updateField('name', e.target.value)}
        placeholder="Name"
      />
```

```
      <input
        value={user.email}
        onChange={(e) => updateField('email', e.target.value)}
        placeholder="Email"
      />
      <input
        type="number"
        value={user.age}
        onChange={(e) => updateField('age', Number(e.target.value))}
        placeholder="Age"
      />
    </form>
  );
}
```

State Update Rules

```
+-------------------------------------------------------------+
|                    STATE UPDATE RULES                        |
+-------------------------------------------------------------+
|                                                             |
|  1. NEVER mutate state directly                             |
|     BAD:  state.count = 5                                   |
|     GOOD: setState({ count: 5 })                            |
|                                                             |
|  2. State updates may be BATCHED (asynchronous)             |
|     React may batch multiple setState calls into one render |
|                                                             |
|  3. Use FUNCTIONAL updates when new state depends on old    |
|     BAD:  setCount(count + 1)  // stale closure risk        |
|     GOOD: setCount(prev => prev + 1)                        |
|                                                             |
|  4. State updates trigger RE-RENDERS                        |
|     The component and its children re-render                |
|                                                             |
|  5. State is PRESERVED between renders                      |
|     React keeps state tied to the component's position      |
|     in the tree                                             |
+-------------------------------------------------------------+
```

4.6 Component Lifecycle

What Is the Component Lifecycle?

Every React component goes through a lifecycle: it is created (mounted), updated (when state or props change), and eventually removed (unmounted). Understanding the lifecycle is essential for managing side effects, data fetching, and cleanup.

Lifecycle Phases

```
+-------------------------------------------------------------+
|                  COMPONENT LIFECYCLE PHASES                   |
+-------------------------------------------------------------+
|                                                              |
|  MOUNTING (Component is created and inserted into DOM)       |
|  +-------------------------------------------------------+ |
|  | constructor() -> render() -> componentDidMount()        | |
|  | [Hooks: useState initializer, useEffect(fn, [])]       | |
|  +-------------------------------------------------------+ |
|                          |                                   |
|                          v                                   |
|  UPDATING (State or props change)                            |
|  +-------------------------------------------------------+ |
|  | render() -> componentDidUpdate()                        | |
|  | [Hooks: useEffect(fn, [deps])]                         | |
|  +-------------------------------------------------------+ |
|                          |                                   |
|                          v                                   |
|  UNMOUNTING (Component is removed from DOM)                  |
|  +-------------------------------------------------------+ |
|  | componentWillUnmount()                                  | |
|  | [Hooks: useEffect cleanup function]                    | |
|  +-------------------------------------------------------+ |
+-------------------------------------------------------------+
```

Class Component Lifecycle Methods

```
+-------------------------------------------------------------+
|              CLASS COMPONENT LIFECYCLE METHODS                |
+-------------------------------------------------------------+
|                                                              |
|  MOUNTING:                                                   |
|    constructor(props)          Initialize state               |
|    static getDerivedStateFromProps(props, state)             |
|    render()                    Return JSX                     |
```

```
|    componentDidMount()         API calls, subscriptions      |
|                                                              |
|  UPDATING:                                                   |
|    static getDerivedStateFromProps(props, state)             |
|    shouldComponentUpdate(nextProps, nextState)                |
|    render()                    Return updated JSX             |
|    getSnapshotBeforeUpdate(prevProps, prevState)              |
|    componentDidUpdate(prevProps, prevState, snapshot)         |
|                                                              |
|  UNMOUNTING:                                                 |
|    componentWillUnmount()      Cleanup timers, listeners     |
|                                                              |
|  ERROR HANDLING:                                             |
|    static getDerivedStateFromError(error)                    |
|    componentDidCatch(error, info)                            |
+--------------------------------------------------------------+
```

Lifecycle with Hooks (Functional Components)

Hooks map to lifecycle methods as follows:

Class Lifecycle Method	Hook Equivalent
`constructor`	`useState` (initialization)
`componentDidMount`	`useEffect(() => { ... }, [])`
`componentDidUpdate`	`useEffect(() => { ... }, [deps])`
`componentWillUnmount`	`useEffect(() => { return () => cleanup }, [])`
`shouldComponentUpdate`	`React.memo()`

Real-World Lifecycle Example

JavaScript -- Functional Component with Hooks:

```javascript
import React, { useState, useEffect } from 'react';

function UserDashboard({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // componentDidMount + componentDidUpdate (when userId changes)
  useEffect(() => {
```

```
  let isCancelled = false;

  async function fetchUser() {
    setLoading(true);
    setError(null);

    try {
      const response = await fetch(`/api/users/${userId}`);
      const data = await response.json();

      if (!isCancelled) {
        setUser(data);
        setLoading(false);
      }
    } catch (err) {
      if (!isCancelled) {
        setError(err.message);
        setLoading(false);
      }
    }
  }

  fetchUser();

  // Cleanup -- componentWillUnmount / before next effect run
  return () => {
    isCancelled = true;
  };
}, [userId]); // Re-run when userId changes

// componentDidMount -- set up event listener
useEffect(() => {
  const handleResize = () => {
    console.log('Window resized');
  };

  window.addEventListener('resize', handleResize);

  // Cleanup -- componentWillUnmount
  return () => {
    window.removeEventListener('resize', handleResize);
  };
}, []); // Empty deps = run once on mount

if (loading) return <div className="spinner">Loading...</div>;
if (error) return <div className="error">Error: {error}</div>;
if (!user) return null;
```

```
  return (
    <div className="dashboard">
      <h1>Welcome, {user.name}</h1>
      <p>Email: {user.email}</p>
      <p>Role: {user.role}</p>
    </div>
  );
}
```

TypeScript -- Same Component:

```
import React, { useState, useEffect } from 'react';

interface User {
  id: number;
  name: string;
  email: string;
  role: string;
}

interface UserDashboardProps {
  userId: number;
}

function UserDashboard({ userId }: UserDashboardProps): React.ReactElement {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let isCancelled = false;

    async function fetchUser(): Promise<void> {
      setLoading(true);
      setError(null);

      try {
        const response = await fetch(`/api/users/${userId}`);
        const data: User = await response.json();

        if (!isCancelled) {
          setUser(data);
          setLoading(false);
        }
      } catch (err) {
        if (!isCancelled) {
```

```
          setError((err as Error).message);
          setLoading(false);
        }
      }
    }

    fetchUser();

    return () => {
      isCancelled = true;
    };
  }, [userId]);

  useEffect(() => {
    const handleResize = (): void => {
      console.log('Window resized');
    };

    window.addEventListener('resize', handleResize);

    return () => {
      window.removeEventListener('resize', handleResize);
    };
  }, []);

  if (loading) return <div className="spinner">Loading...</div>;
  if (error) return <div className="error">Error: {error}</div>;
  if (!user) return <></>;

  return (
    <div className="dashboard">
      <h1>Welcome, {user.name}</h1>
      <p>Email: {user.email}</p>
      <p>Role: {user.role}</p>
    </div>
  );
}
```

Error Boundaries (Class Components Only)

Error boundaries catch JavaScript errors in child components and display a fallback UI. They must be class components.

```
// TypeScript Error Boundary
import React, { Component, ErrorInfo, ReactNode } from 'react';
```

```
interface ErrorBoundaryProps {
  children: ReactNode;
  fallback?: ReactNode;
}

interface ErrorBoundaryState {
  hasError: boolean;
  error: Error | null;
}

class ErrorBoundary extends Component<ErrorBoundaryProps, ErrorBoundaryState> {
  constructor(props: ErrorBoundaryProps) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error: Error): ErrorBoundaryState {
    return { hasError: true, error };
  }

  componentDidCatch(error: Error, errorInfo: ErrorInfo): void {
    console.error('Error Boundary caught:', error, errorInfo);
    // Log to error reporting service (e.g., Sentry)
  }

  render(): ReactNode {
    if (this.state.hasError) {
      return this.props.fallback || (
        <div className="error-boundary">
          <h2>Something went wrong</h2>
          <p>{this.state.error?.message}</p>
          <button onClick={() => this.setState({ hasError: false, error: null
})}>
            Try Again
          </button>
        </div>
      );
    }

    return this.props.children;
  }
}

// Usage
<ErrorBoundary fallback={<div>Oops! Something broke.</div>}>
```

```
  <UserDashboard userId={1} />
</ErrorBoundary>
```

Part 5: React Hooks Deep Dive

5.1 Introduction to Hooks

What Are Hooks?

Hooks are special functions introduced in React 16.8 that let you use state, lifecycle, and other React features in functional components -- without writing classes.

Why Hooks Were Introduced

Problem with Classes	How Hooks Solve It
Complex lifecycle methods mixing unrelated logic	Each hook handles one concern
Hard to reuse stateful logic between components	Custom hooks enable logic sharing
`this` keyword confusion and binding issues	No `this` -- just plain functions
Verbose boilerplate (constructor, bind, render)	Concise, focused code
Difficult to optimize (class instances)	Pure functions, easier tree shaking

Rules of Hooks

```
+-------------------------------------------------------------+
|                       RULES OF HOOKS                         |
+-------------------------------------------------------------+
|                                                             |
|  1. Only call hooks at the TOP LEVEL                        |
|     - Never inside loops, conditions, or nested functions   |
|     - React relies on call ORDER to track state             |
|                                                             |
|  2. Only call hooks from REACT FUNCTIONS                    |
|     - Functional components                                 |
|     - Custom hooks (functions starting with "use")          |
|     - Never from regular JavaScript functions               |
|                                                             |
|  WHY? React tracks hooks by their call order:               |
|                                                             |
|  First render:    hook1 -> hook2 -> hook3                   |
|  Second render:   hook1 -> hook2 -> hook3  (same order!)    |
|                                                             |
```

```
|  If order changes, React cannot match state correctly!       |
+-------------------------------------------------------------+
```

Hook Lifecycle Overview

```
+-------------------------------------------------------------+
|                    HOOK LIFECYCLE                            |
+-------------------------------------------------------------+
|                                                             |
|  INITIAL RENDER                                             |
|  +-------------------------------------------------------+ |
|  | 1. useState(initialValue) -> creates state             | |
|  | 2. Component renders (returns JSX)                     | |
|  | 3. React updates DOM                                   | |
|  | 4. useEffect callbacks run (after paint)               | |
|  +-------------------------------------------------------+ |
|                          |                                  |
|                   State Changes                             |
|                          |                                  |
|  RE-RENDER                                                  |
|  +-------------------------------------------------------+ |
|  | 1. useState returns current state (not initial)        | |
|  | 2. Component re-renders with new state                 | |
|  | 3. React diffs and updates DOM                         | |
|  | 4. useEffect cleanup runs (if deps changed)           | |
|  | 5. useEffect callbacks run (if deps changed)          | |
|  +-------------------------------------------------------+ |
|                          |                                  |
|                   Component Removed                         |
|                          |                                  |
|  UNMOUNT                                                    |
|  +-------------------------------------------------------+ |
|  | 1. useEffect cleanup functions run                     | |
|  | 2. State is discarded                                  | |
|  +-------------------------------------------------------+ |
+-------------------------------------------------------------+
```

5.2 useState

What It Does

`useState` declares a state variable in a functional component. It returns a pair: the current state value and a function to update it.

Syntax

```
const [state, setState] = useState<Type>(initialValue);
```

How It Works Internally

1. On first render, React stores the initial value in an internal array at a specific index
2. On re-renders, React returns the stored value (not the initial value)
3. When `setState` is called, React schedules a re-render with the new value
4. React batches multiple `setState` calls for performance

When to Use

- Tracking user input (form values)
- Toggle visibility (modals, dropdowns)
- Counters and numerical values
- Any data that changes over time and affects rendering

JavaScript Example

```
import React, { useState } from 'react';

function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');

  const addTodo = () => {
    if (!input.trim()) return;
    setTodos(prev => [...prev, { id: Date.now(), text: input, done: false }]);
    setInput('');
  };

  const toggleTodo = (id) => {
    setTodos(prev =>
      prev.map(todo =>
        todo.id === id ? { ...todo, done: !todo.done } : todo
      )
    );
  };
```

```
  const deleteTodo = (id) => {
    setTodos(prev => prev.filter(todo => todo.id !== id));
  };

  return (
    <div className="todo-app">
      <h2>Todo List</h2>
      <div className="input-group">
        <input
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => e.key === 'Enter' && addTodo()}
          placeholder="Add a todo..."
        />
        <button onClick={addTodo}>Add</button>
      </div>
      <ul>
        {todos.map(todo => (
          <li key={todo.id} className={todo.done ? 'completed' : ''}>
            <span onClick={() => toggleTodo(todo.id)}>{todo.text}</span>
            <button onClick={() => deleteTodo(todo.id)}>Delete</button>
          </li>
        ))}
      </ul>
      <p>{todos.filter(t => !t.done).length} items remaining</p>
    </div>
  );
}
```

TypeScript Example

```
import React, { useState } from 'react';

interface Todo {
  id: number;
  text: string;
  done: boolean;
}

function TodoApp(): React.ReactElement {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [input, setInput] = useState<string>('');

  const addTodo = (): void => {
```

```
    if (!input.trim()) return;
    setTodos(prev => [...prev, { id: Date.now(), text: input, done: false }]);
    setInput('');
  };

  const toggleTodo = (id: number): void => {
    setTodos(prev =>
      prev.map(todo =>
        todo.id === id ? { ...todo, done: !todo.done } : todo
      )
    );
  };

  const deleteTodo = (id: number): void => {
    setTodos(prev => prev.filter(todo => todo.id !== id));
  };

  return (
    <div className="todo-app">
      <h2>Todo List</h2>
      <div className="input-group">
        <input
          value={input}
          onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
setInput(e.target.value)}
          onKeyDown={(e: React.KeyboardEvent) => e.key === 'Enter' &&
addTodo()}
          placeholder="Add a todo..."
        />
        <button onClick={addTodo}>Add</button>
      </div>
      <ul>
        {todos.map(todo => (
          <li key={todo.id} className={todo.done ? 'completed' : ''}>
            <span onClick={() => toggleTodo(todo.id)}>{todo.text}</span>
            <button onClick={() => deleteTodo(todo.id)}>Delete</button>
          </li>
        ))}
      </ul>
      <p>{todos.filter(t => !t.done).length} items remaining</p>
    </div>
  );
}
```

Lazy Initialization

When the initial value is expensive to compute, pass a function:

```
// Expensive computation runs on EVERY render
const [data, setData] = useState(expensiveComputation()); // BAD

// Function runs ONLY on first render
const [data, setData] = useState(() => expensiveComputation()); // GOOD

// Real-world example: reading from localStorage
const [theme, setTheme] = useState<string>(() => {
  return localStorage.getItem('theme') || 'light';
});
```

5.3 useEffect

What It Does

`useEffect` lets you perform side effects in functional components. Side effects include data fetching, subscriptions, timers, DOM manipulation, and logging.

Why It Exists

Functional components are pure rendering functions -- they should only compute JSX based on props and state. Side effects need a dedicated mechanism, which `useEffect` provides.

How It Works Internally

1. After React renders the component and updates the DOM, it runs the effect
2. Effects run asynchronously (after browser paint), so they don't block rendering
3. Before running the next effect (on re-render), React runs the cleanup function from the previous effect
4. On unmount, React runs the final cleanup

When to Use

Use Case	Example
Data fetching	Fetch data from API on mount or when dependencies change
Subscriptions	WebSocket connections, event listeners
Timers	setInterval, setTimeout

DOM manipulation	Focus input, scroll to element
Synchronizing	Sync state with external systems (localStorage, URL)

Three Forms of useEffect

```
+-------------------------------------------------------------+
|              THREE FORMS OF useEffect                        |
+-------------------------------------------------------------+
|                                                              |
|  FORM 1: Run on EVERY render                                |
|  useEffect(() => {                                           |
|    // runs after every render                                |
|  });                                                         |
|  Use case: Logging, debugging                                |
|                                                              |
|  FORM 2: Run ONCE on mount                                  |
|  useEffect(() => {                                           |
|    // runs only once after initial render                    |
|    return () => { /* cleanup on unmount */ };                |
|  }, []);                                                     |
|  Use case: Initial data fetch, event listeners               |
|                                                              |
|  FORM 3: Run when DEPENDENCIES change                        |
|  useEffect(() => {                                           |
|    // runs when dep1 or dep2 change                          |
|    return () => { /* cleanup before re-run */ };             |
|  }, [dep1, dep2]);                                           |
|  Use case: Re-fetch when ID changes, re-subscribe           |
+-------------------------------------------------------------+
```

useEffect Execution Flow

```
Component Renders
       |
       v
React updates DOM
       |
       v
Browser paints screen
       |
       v
Run useEffect cleanup (from previous render, if deps changed)
       |
```

```
       v
Run useEffect callback (if deps changed or first render)
       |
       v
Component re-renders (if state updated inside effect)
```

Real-World Example: Data Fetching with Cleanup

```typescript
import React, { useState, useEffect } from 'react';

interface Product {
  id: number;
  name: string;
  price: number;
  image: string;
}

function ProductList({ categoryId }: { categoryId: number }) {
  const [products, setProducts] = useState<Product[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const controller = new AbortController(); // For cleanup

    async function fetchProducts(): Promise<void> {
      setLoading(true);
      setError(null);

      try {
        const response = await fetch(
          `/api/categories/${categoryId}/products`,
          { signal: controller.signal }
        );

        if (!response.ok) throw new Error('Failed to fetch');

        const data: Product[] = await response.json();
        setProducts(data);
      } catch (err) {
        if ((err as Error).name !== 'AbortError') {
          setError((err as Error).message);
        }
      } finally {
        setLoading(false);
```

```
      }
    }

    fetchProducts();

    // Cleanup: abort fetch if component unmounts or categoryId changes
    return () => controller.abort();
  }, [categoryId]); // Re-fetch when category changes

  if (loading) return <div>Loading products...</div>;
  if (error) return <div>Error: {error}</div>;

  return (
    <div className="product-grid">
      {products.map(product => (
        <div key={product.id} className="product-card">
          <img src={product.image} alt={product.name} />
          <h3>{product.name}</h3>
          <p>${product.price.toFixed(2)}</p>
        </div>
      ))}
    </div>
  );
}
```

Real-World Example: WebSocket Subscription

```
useEffect(() => {
  const ws = new WebSocket('wss://api.example.com/notifications');

  ws.onmessage = (event: MessageEvent) => {
    const notification = JSON.parse(event.data);
    setNotifications(prev => [notification, ...prev]);
  };

  ws.onerror = (error: Event) => {
    console.error('WebSocket error:', error);
  };

  // Cleanup: close connection on unmount
  return () => {
    ws.close();
```

```
  };
}, []);
```

5.4 useContext

What It Does

`useContext` provides access to React Context values without nesting Consumer components. It allows sharing data across the component tree without prop drilling.

Why It Exists

Passing data through many layers of props (prop drilling) makes code verbose and hard to maintain. `useContext` provides a cleaner alternative for data that many components need (e.g., theme, user, locale).

How It Works Internally

1. A Context is created with `createContext`
2. A Provider component wraps the tree, supplying the value
3. Any descendant component can call `useContext` to read the value
4. When the Provider's value changes, all consumers re-render

When to Use

- Theme (dark/light mode)
- Authenticated user data
- Locale/language preferences
- Configuration settings

When NOT to Use

- For frequently changing data (every keystroke) -- use state management instead
- When only one or two levels of nesting exist -- just pass props

Real-World Example: Theme Context

```
// ThemeContext.tsx
import React, { createContext, useContext, useState, ReactNode } from 'react';

type Theme = 'light' | 'dark';
```

```
interface ThemeContextType {
  theme: Theme;
  toggleTheme: () => void;
  colors: {
    background: string;
    text: string;
    primary: string;
  };
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<Theme>('light');

  const toggleTheme = (): void => {
    setTheme(prev => (prev === 'light' ? 'dark' : 'light'));
  };

  const colors = {
    background: theme === 'light' ? '#ffffff' : '#1a1a2e',
    text: theme === 'light' ? '#333333' : '#e0e0e0',
    primary: theme === 'light' ? '#3f51b5' : '#7c4dff',
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme, colors }}>
      {children}
    </ThemeContext.Provider>
  );
}

// Custom hook for consuming the context
export function useTheme(): ThemeContextType {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme must be used within a ThemeProvider');
  }
  return context;
}
```

```
// Usage in components
function Header() {
  const { theme, toggleTheme, colors } = useTheme();
```

```
  return (
    <header style={{ background: colors.background, color: colors.text }}>
      <h1>My App</h1>
      <button onClick={toggleTheme}>
        Switch to {theme === 'light' ? 'Dark' : 'Light'} Mode
      </button>
    </header>
  );
}

// App.tsx
function App() {
  return (
    <ThemeProvider>
      <Header />
      <MainContent />
      <Footer />
    </ThemeProvider>
  );
}
```

5.5 useReducer

What It Does

`useReducer` is an alternative to `useState` for managing complex state logic. It accepts a reducer function and an initial state, returning the current state and a dispatch function.

Why It Exists

When state logic involves multiple sub-values, depends on previous state, or follows complex transitions, `useReducer` is more predictable than multiple `useState` calls.

When to Use

Scenario	Use useState	Use useReducer
Simple values (toggle, counter)	Yes	Overkill
Related state fields that update together	Possible	Preferred
Complex state transitions	Messy	Clean
State logic shared between components	Difficult	Easy (extract reducer)

Predictable state machine behavior	No	Yes

How It Works Internally

```
+-------------------------------------------------------------+
|                    useReducer FLOW                          |
+-------------------------------------------------------------+
|                                                             |
|  Component dispatches an ACTION                             |
|       |                                                     |
|       v                                                     |
|  REDUCER FUNCTION receives (currentState, action)           |
|       |                                                     |
|       v                                                     |
|  Reducer computes and returns NEW STATE                     |
|       |                                                     |
|       v                                                     |
|  React compares new state with old state                    |
|       |                                                     |
|       +--> Same? -> No re-render                            |
|       |                                                     |
|       +--> Different? -> Component re-renders               |
+-------------------------------------------------------------+
```

Real-World Example: Shopping Cart

```typescript
import React, { useReducer } from 'react';

// Types
interface CartItem {
  id: number;
  name: string;
  price: number;
  quantity: number;
}

interface CartState {
  items: CartItem[];
  total: number;
}

// Action types
type CartAction =
```

```
  | { type: 'ADD_ITEM'; payload: Omit<CartItem, 'quantity'> }
  | { type: 'REMOVE_ITEM'; payload: { id: number } }
  | { type: 'UPDATE_QUANTITY'; payload: { id: number; quantity: number } }
  | { type: 'CLEAR_CART' };

// Initial state
const initialState: CartState = { items: [], total: 0 };

// Helper function
function calculateTotal(items: CartItem[]): number {
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}

// Reducer function -- pure function, no side effects
function cartReducer(state: CartState, action: CartAction): CartState {
  switch (action.type) {
    case 'ADD_ITEM': {
      const existing = state.items.find(item => item.id === action.payload.id);
      let newItems: CartItem[];

      if (existing) {
        newItems = state.items.map(item =>
          item.id === action.payload.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      } else {
        newItems = [...state.items, { ...action.payload, quantity: 1 }];
      }

      return { items: newItems, total: calculateTotal(newItems) };
    }

    case 'REMOVE_ITEM': {
      const newItems = state.items.filter(item => item.id !==
action.payload.id);
      return { items: newItems, total: calculateTotal(newItems) };
    }

    case 'UPDATE_QUANTITY': {
      const newItems = state.items.map(item =>
        item.id === action.payload.id
          ? { ...item, quantity: action.payload.quantity }
          : item
      ).filter(item => item.quantity > 0);
      return { items: newItems, total: calculateTotal(newItems) };
    }
```

```
    case 'CLEAR_CART':
      return initialState;

    default:
      return state;
  }
}

// Component
function ShoppingCart(): React.ReactElement {
  const [state, dispatch] = useReducer(cartReducer, initialState);

  return (
    <div className="cart">
      <h2>Shopping Cart ({state.items.length} items)</h2>
      {state.items.map(item => (
        <div key={item.id} className="cart-item">
          <span>{item.name}</span>
          <span>${item.price.toFixed(2)}</span>
          <input
            type="number"
            value={item.quantity}
            min={0}
            onChange={(e) =>
              dispatch({
                type: 'UPDATE_QUANTITY',
                payload: { id: item.id, quantity: Number(e.target.value) }
              })
            }
          />
          <button onClick={() => dispatch({ type: 'REMOVE_ITEM', payload: { id:
item.id } })}>
            Remove
          </button>
        </div>
      ))}
      <div className="cart-total">
        <strong>Total: ${state.total.toFixed(2)}</strong>
      </div>
      <button onClick={() => dispatch({ type: 'CLEAR_CART' })}>
        Clear Cart
      </button>
    </div>
  );
}
```

5.6 useMemo

What It Does

`useMemo` caches the result of an expensive computation and only recomputes it when its dependencies change.

Why It Exists

Without `useMemo`, expensive calculations re-run on every render, even when their inputs haven't changed. This can cause performance issues in large applications.

When to Use

- Expensive computations (sorting, filtering large arrays)
- Deriving data from state/props
- Preventing unnecessary re-creation of objects/arrays passed as props

When NOT to Use

- Simple calculations (addition, string concatenation)
- When the dependency changes on every render anyway
- Premature optimization -- measure first, then optimize

How It Works Internally

```
Render 1: deps = [a, b]
  -> Compute result
  -> Cache: { deps: [a, b], result: value1 }

Render 2: deps = [a, b]  (same)
  -> Return cached value1 (no recomputation)

Render 3: deps = [a, c]  (changed!)
  -> Recompute result
  -> Cache: { deps: [a, c], result: value2 }
```

Real-World Example

```
import React, { useState, useMemo } from 'react';

interface Employee {
```

```
  id: number;
  name: string;
  department: string;
  salary: number;
}

function EmployeeTable({ employees }: { employees: Employee[] }) {
  const [search, setSearch] = useState<string>('');
  const [sortBy, setSortBy] = useState<keyof Employee>('name');
  const [filterDept, setFilterDept] = useState<string>('all');

  // EXPENSIVE: filtering and sorting a large list
  // useMemo ensures this only recomputes when search, sortBy, or filterDept
change
  const processedEmployees = useMemo(() => {
    console.log('Recomputing employee list...'); // For debugging

    let result = [...employees];

    // Filter by department
    if (filterDept !== 'all') {
      result = result.filter(emp => emp.department === filterDept);
    }

    // Filter by search
    if (search) {
      const searchLower = search.toLowerCase();
      result = result.filter(emp =>
        emp.name.toLowerCase().includes(searchLower)
      );
    }

    // Sort
    result.sort((a, b) => {
      if (a[sortBy] < b[sortBy]) return -1;
      if (a[sortBy] > b[sortBy]) return 1;
      return 0;
    });

    return result;
  }, [employees, search, sortBy, filterDept]);

  // Memoize statistics
  const stats = useMemo(() => ({
    total: processedEmployees.length,
    avgSalary: processedEmployees.length > 0
      ? processedEmployees.reduce((sum, e) => sum + e.salary, 0) /
processedEmployees.length
```

```
      : 0,
    maxSalary: Math.max(...processedEmployees.map(e => e.salary), 0)
  }), [processedEmployees]);

  return (
    <div>
      <input
        value={search}
        onChange={e => setSearch(e.target.value)}
        placeholder="Search employees..."
      />
      <select value={filterDept} onChange={e => setFilterDept(e.target.value)}>
        <option value="all">All Departments</option>
        <option value="engineering">Engineering</option>
        <option value="sales">Sales</option>
      </select>
      <div>
        <p>Showing {stats.total} employees | Avg Salary:
${stats.avgSalary.toFixed(0)}</p>
      </div>
      <table>
        <thead>
          <tr>
            <th onClick={() => setSortBy('name')}>Name</th>
            <th onClick={() => setSortBy('department')}>Department</th>
            <th onClick={() => setSortBy('salary')}>Salary</th>
          </tr>
        </thead>
        <tbody>
          {processedEmployees.map(emp => (
            <tr key={emp.id}>
              <td>{emp.name}</td>
              <td>{emp.department}</td>
              <td>${emp.salary.toLocaleString()}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

5.7 useCallback

What It Does

`useCallback` caches a function definition so that it maintains referential equality between renders unless its dependencies change.

Why It Exists

In JavaScript, functions created inside a component are new objects on every render. When passed as props to child components wrapped in `React.memo`, new function references cause unnecessary re-renders. `useCallback` preserves the same function reference.

useMemo vs useCallback

Feature	useMemo	useCallback
Caches	Computed value	Function definition
Returns	The result of calling the function	The function itself
Syntax	`useMemo(() => computeValue(), [deps])`	`useCallback((args) => doSomething(args), [deps])`
Use Case	Expensive calculations	Functions passed as props

```
// These are equivalent:
const memoizedFn = useCallback((a, b) => a + b, []);
const memoizedFn = useMemo(() => (a, b) => a + b, []);
```

Real-World Example

```
import React, { useState, useCallback, memo } from 'react';

// Child component wrapped in React.memo
// Only re-renders when its props actually change
const TodoItem = memo(function TodoItem({
  todo,
  onToggle,
  onDelete
}: {
  todo: { id: number; text: string; done: boolean };
  onToggle: (id: number) => void;
```

```
  onDelete: (id: number) => void;
}) {
  console.log(`Rendering TodoItem: ${todo.text}`); // Debug

  return (
    <li className={todo.done ? 'completed' : ''}>
      <span onClick={() => onToggle(todo.id)}>{todo.text}</span>
      <button onClick={() => onDelete(todo.id)}>Delete</button>
    </li>
  );
});

function TodoList() {
  const [todos, setTodos] = useState([
    { id: 1, text: 'Learn React', done: false },
    { id: 2, text: 'Build project', done: false }
  ]);
  const [input, setInput] = useState('');

  // Without useCallback: new function on every render
  // -> TodoItem re-renders even when its todo hasn't changed
  // With useCallback: same function reference between renders
  // -> TodoItem skips re-render if its todo hasn't changed

  const handleToggle = useCallback((id: number): void => {
    setTodos(prev =>
      prev.map(todo =>
        todo.id === id ? { ...todo, done: !todo.done } : todo
      )
    );
  }, []);

  const handleDelete = useCallback((id: number): void => {
    setTodos(prev => prev.filter(todo => todo.id !== id));
  }, []);

  return (
    <div>
      <input value={input} onChange={e => setInput(e.target.value)} />
      <ul>
        {todos.map(todo => (
          <TodoItem
            key={todo.id}
            todo={todo}
            onToggle={handleToggle}
            onDelete={handleDelete}
          />
        ))}
```

```
      </ul>
    </div>
  );
}
```

5.8 useRef

What It Does

`useRef` creates a mutable reference object that persists for the entire lifetime of the component. It does NOT cause re-renders when its value changes.

Why It Exists

Sometimes you need to:

- Access DOM elements directly (focus, scroll, measure)
- Store mutable values that shouldn't trigger re-renders (timers, previous values)
- Keep a reference across renders without re-rendering

When to Use

Use Case	Example
DOM access	Focus input, scroll into view, measure size
Previous value	Track previous state for comparisons
Mutable instance variable	Timer IDs, counters, flags
Storing external library instances	Chart.js, D3, map libraries

How It Works

```
+-------------------------------------------------------------+
|                        useRef BEHAVIOR                       |
+-------------------------------------------------------------+
|                                                             |
|  const ref = useRef(initialValue);                          |
|                                                             |
|  ref.current = initialValue  (mutable, persists)            |
|                                                             |
|  Render 1: ref.current = 'A'                                |
```

```
|  Render 2: ref.current = 'A'  (still 'A', not reset)        |
|  Update:   ref.current = 'B'  (no re-render triggered!)     |
|  Render 3: ref.current = 'B'  (value persisted)             |
|                                                              |
|  KEY DIFFERENCE from useState:                               |
|  - useState: value change -> re-render                       |
|  - useRef: value change -> NO re-render                      |
+--------------------------------------------------------------+
```

Real-World Examples

```
import React, { useRef, useState, useEffect } from 'react';

// Example 1: DOM Access -- Auto-focus input
function SearchBar() {
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    inputRef.current?.focus(); // Focus on mount
  }, []);

  return <input ref={inputRef} placeholder="Search..." />;
}

// Example 2: Timer management
function Stopwatch() {
  const [time, setTime] = useState<number>(0);
  const [isRunning, setIsRunning] = useState<boolean>(false);
  const intervalRef = useRef<NodeJS.Timeout | null>(null);

  const start = (): void => {
    if (isRunning) return;
    setIsRunning(true);
    intervalRef.current = setInterval(() => {
      setTime(prev => prev + 10);
    }, 10);
  };

  const stop = (): void => {
    if (intervalRef.current) clearInterval(intervalRef.current);
    setIsRunning(false);
  };

  const reset = (): void => {
    stop();
```

```
    setTime(0);
  };

  // Cleanup on unmount
  useEffect(() => {
    return () => {
      if (intervalRef.current) clearInterval(intervalRef.current);
    };
  }, []);

  const formatTime = (ms: number): string => {
    const minutes = Math.floor(ms / 60000);
    const seconds = Math.floor((ms % 60000) / 1000);
    const centiseconds = Math.floor((ms % 1000) / 10);
    return `${minutes.toString().padStart(2,
'0')}:${seconds.toString().padStart(2,
'0')}.${centiseconds.toString().padStart(2, '0')}`;
  };

  return (
    <div className="stopwatch">
      <h1>{formatTime(time)}</h1>
      <button onClick={start} disabled={isRunning}>Start</button>
      <button onClick={stop} disabled={!isRunning}>Stop</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}

// Example 3: Tracking previous value
function usePrevious<T>(value: T): T | undefined {
  const ref = useRef<T | undefined>(undefined);

  useEffect(() => {
    ref.current = value;
  }, [value]);

  return ref.current;
}

function PriceDisplay({ price }: { price: number }) {
  const previousPrice = usePrevious(price);

  return (
    <div>
      <p>Current: ${price}</p>
      <p>Previous: ${previousPrice ?? 'N/A'}</p>
      {previousPrice !== undefined && (
```

```
        <span className={price > previousPrice ? 'up' : 'down'}>
          {price > previousPrice ? 'UP' : 'DOWN'}
        </span>
      )}
    </div>
  );
}
```

5.9 Custom Hooks

What Are Custom Hooks?

Custom hooks are JavaScript/TypeScript functions whose names start with `use` and that can call other hooks. They let you extract reusable stateful logic from components.

Why Create Custom Hooks?

- **Reuse logic** across multiple components without duplication
- **Separation of concerns** -- keep components focused on rendering
- **Testable** -- hook logic can be tested independently
- **Composition** -- combine multiple hooks into one

Real-World Custom Hooks

1. useFetch -- Generic data fetching hook:

```
import { useState, useEffect } from 'react';

interface UseFetchResult<T> {
  data: T | null;
  loading: boolean;
  error: string | null;
  refetch: () => void;
}

function useFetch<T>(url: string): UseFetchResult<T> {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);
  const [trigger, setTrigger] = useState<number>(0);

  const refetch = (): void => setTrigger(prev => prev + 1);
```

```
  useEffect(() => {
    const controller = new AbortController();

    async function fetchData(): Promise<void> {
      setLoading(true);
      setError(null);

      try {
        const response = await fetch(url, { signal: controller.signal });
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const result: T = await response.json();
        setData(result);
      } catch (err) {
        if ((err as Error).name !== 'AbortError') {
          setError((err as Error).message);
        }
      } finally {
        setLoading(false);
      }
    }

    fetchData();
    return () => controller.abort();
  }, [url, trigger]);

  return { data, loading, error, refetch };
}

// Usage
function UserList() {
  const { data: users, loading, error, refetch } = useFetch<User[]>
('/api/users');

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error} <button onClick=
{refetch}>Retry</button></div>;

  return (
    <ul>
      {users?.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
}
```

2. useDebounce -- Debounce rapidly changing values:

```
import { useState, useEffect } from 'react';

function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}

// Usage: Search with debounce
function SearchPage() {
  const [query, setQuery] = useState<string>('');
  const debouncedQuery = useDebounce(query, 300);

  const { data: results } = useFetch<SearchResult[]>(
    debouncedQuery ? `/api/search?q=${debouncedQuery}` : ''
  );

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search..."
      />
      {results?.map(r => <div key={r.id}>{r.title}</div>)}
    </div>
  );
}
```

3. useLocalStorage -- Persist state in localStorage:

```
import { useState, useEffect } from 'react';

function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T) =>
void] {
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
```

```
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.error('Error saving to localStorage:', error);
    }
  }, [key, storedValue]);

  return [storedValue, setStoredValue];
}

// Usage
function Settings() {
  const [theme, setTheme] = useLocalStorage<'light' | 'dark'>('theme',
'light');
  const [fontSize, setFontSize] = useLocalStorage<number>('fontSize', 14);

  return (
    <div>
      <select value={theme} onChange={e => setTheme(e.target.value as 'light' |
'dark')}>
        <option value="light">Light</option>
        <option value="dark">Dark</option>
      </select>
      <input
        type="range"
        min={12}
        max={24}
        value={fontSize}
        onChange={e => setFontSize(Number(e.target.value))}
      />
    </div>
  );
}
```

4. useOnClickOutside -- Detect clicks outside an element:

```
import { useEffect, RefObject } from 'react';

function useOnClickOutside(ref: RefObject<HTMLElement>, handler: () => void):
void {
```

```
  useEffect(() => {
    const listener = (event: MouseEvent | TouchEvent): void => {
      if (!ref.current || ref.current.contains(event.target as Node)) {
        return;
      }
      handler();
    };

    document.addEventListener('mousedown', listener);
    document.addEventListener('touchstart', listener);

    return () => {
      document.removeEventListener('mousedown', listener);
      document.removeEventListener('touchstart', listener);
    };
  }, [ref, handler]);
}

// Usage: Close dropdown on outside click
function Dropdown() {
  const [isOpen, setIsOpen] = useState(false);
  const dropdownRef = useRef<HTMLDivElement>(null);

  useOnClickOutside(dropdownRef, () => setIsOpen(false));

  return (
    <div ref={dropdownRef}>
      <button onClick={() => setIsOpen(!isOpen)}>Menu</button>
      {isOpen && (
        <ul className="dropdown-menu">
          <li>Profile</li>
          <li>Settings</li>
          <li>Logout</li>
        </ul>
      )}
    </div>
  );
}
```

Custom Hooks Summary

Hook	Purpose	Key Dependencies
`useFetch`	Data fetching with loading/error states	URL
`useDebounce`	Delay rapidly changing values	Value, delay

`useLocalStorage`	Persist state in localStorage	Key, initial value
`useOnClickOutside`	Detect clicks outside element	Ref, handler
`usePrevious`	Track the previous render's value	Value
`useMediaQuery`	Respond to CSS media query matches	Query string
`useToggle`	Boolean toggle state	Initial value

Part 6: State Management

6.1 Understanding State in React Applications

State Categories

React applications deal with multiple categories of state:

```
+-------------------------------------------------------------+
|                    TYPES OF STATE                            |
+-------------------------------------------------------------+
|                                                             |
|  LOCAL STATE                                                |
|  +-------------------------------------------------------+ |
|  | Managed within a single component                     | |
|  | Examples: form input, toggle, counter                 | |
|  | Tool: useState, useReducer                            | |
|  +-------------------------------------------------------+ |
|                                                             |
|  SHARED STATE                                               |
|  +-------------------------------------------------------+ |
|  | Shared between a few related components               | |
|  | Examples: selected tab, filter criteria               | |
|  | Tool: Lift state up to common parent                  | |
|  +-------------------------------------------------------+ |
|                                                             |
|  GLOBAL STATE                                               |
|  +-------------------------------------------------------+ |
|  | Needed across many components at different depths     | |
|  | Examples: auth user, theme, language                  | |
|  | Tool: Context API, Redux, Zustand                     | |
|  +-------------------------------------------------------+ |
|                                                             |
|  SERVER STATE                                               |
|  +-------------------------------------------------------+ |
|  | Data from external APIs (async, cached, stale)        | |
|  | Examples: user list, product catalog                  | |
|  | Tool: React Query, SWR, RTK Query                     | |
|  +-------------------------------------------------------+ |
|                                                             |
|  URL STATE                                                  |
|  +-------------------------------------------------------+ |
|  | State stored in the URL (path, query params)          | |
|  | Examples: search query, page number, selected tab     | |
```

```
|   | Tool: React Router, useSearchParams                        | |
|   +-------------------------------------------------------+ |
+-----------------------------------------------------------------+
```

6.2 Local State

Local state is the simplest form of state management. It belongs to a single component and is managed using `useState` or `useReducer`.

When to Use Local State

- The data is only relevant to one component
- No other component needs to read or update it
- Examples: form inputs, toggle states, local counters

```
function LoginForm() {
  // All state is local to this component
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [errors, setErrors] = useState<Record<string, string>>({});

  const handleSubmit = (e: React.FormEvent): void => {
    e.preventDefault();
    const newErrors: Record<string, string> = {};

    if (!email) newErrors.email = 'Email is required';
    if (!password) newErrors.password = 'Password is required';

    if (Object.keys(newErrors).length > 0) {
      setErrors(newErrors);
      return;
    }

    // Submit logic
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={e => setEmail(e.target.value)}
```

```
        placeholder="Email"
      />
      {errors.email && <span className="error">{errors.email}</span>}

      <div className="password-group">
        <input
          type={showPassword ? 'text' : 'password'}
          value={password}
          onChange={e => setPassword(e.target.value)}
          placeholder="Password"
        />
        <button type="button" onClick={() => setShowPassword(!showPassword)}>
          {showPassword ? 'Hide' : 'Show'}
        </button>
      </div>
      {errors.password && <span className="error">{errors.password}</span>}

      <button type="submit">Login</button>
    </form>
  );
}
```

6.3 The Prop Drilling Problem

What Is Prop Drilling?

Prop drilling occurs when data must be passed through multiple intermediate components that don't use the data themselves -- they just pass it along.

Prop Drilling Visualization

```
+-------------------------------------------------------------+
|                    PROP DRILLING PROBLEM                     |
+-------------------------------------------------------------+
|                                                             |
|  App (has user state)                                       |
|    |                                                        |
|    +-- user={user} --------------------------------+        |
|    |                                              |        |
|    v                                              v        |
|  Layout (doesn't use user, just passes it)      Sidebar     |
|    |                                                        |
|    +-- user={user} --+                                      |
```

```
|    |                 |                                        |
|    v                 v                                        |
|  Header           Content (doesn't use user)                 |
|    |                 |                                        |
|    +-- user={user}   +-- user={user}                         |
|    |                 |                                        |
|    v                 v                                        |
|  UserMenu         Dashboard                                  |
|  (finally uses     |                                         |
|   user!)           +-- user={user}                           |
|                    |                                        |
|                    v                                        |
|                  WelcomeBanner                               |
|                  (finally uses user!)                        |
+-------------------------------------------------------------+
     Problem: Layout, Content, Dashboard don't need user
              but MUST pass it through anyway
```

Why Prop Drilling Is Problematic

Problem	Impact
Verbose code	Every intermediate component must declare and pass the prop
Tight coupling	Intermediate components depend on data they don't use
Refactoring difficulty	Adding/removing a prop requires changes in many files
Maintenance burden	Hard to trace where data flows through the tree

---

6.4 Context API

What Is Context?

React Context API provides a way to share state across the component tree without passing props manually through every level. It creates a "tunnel" that allows any descendant to access the value directly.

How Context Solves Prop Drilling

```
+-------------------------------------------------------------+
|                 CONTEXT API SOLUTION                         |
+-------------------------------------------------------------+
|                                                             |
|  AuthProvider (wraps the tree with user context)            |
```

```
|    |                                                          |
|    v                                                          |
|  App                                                          |
|    |                                                          |
|    +-------------------+                                      |
|    |                   |                                      |
|    v                   v                                      |
|  Layout              Sidebar                                  |
|    |                                                          |
|    +----------+                                               |
|    |          |                                               |
|    v          v                                               |
|  Header    Content                                            |
|    |          |                                               |
|    v          v                                               |
|  UserMenu  Dashboard                                          |
|  useAuth()     |                                              |
|  (direct       v                                              |
|   access!)   WelcomeBanner                                    |
|              useAuth()                                        |
|              (direct access!)                                 |
|                                                               |
|  No intermediate components need to know about user!          |
+---------------------------------------------------------------+
```

Complete Context API Example: Authentication

```
// context/AuthContext.tsx
import React, { createContext, useContext, useState, useEffect, ReactNode }
from 'react';

// Types
interface User {
  id: number;
  name: string;
  email: string;
  role: 'admin' | 'user';
  avatar?: string;
}

interface AuthContextType {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
```

```
  logout: () => void;
  error: string | null;
}

// Create context with undefined default
const AuthContext = createContext<AuthContextType | undefined>(undefined);

// Provider component
export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  // Check for existing session on mount
  useEffect(() => {
    const checkAuth = async (): Promise<void> => {
      try {
        const token = localStorage.getItem('token');
        if (!token) {
          setIsLoading(false);
          return;
        }

        const response = await fetch('/api/auth/me', {
          headers: { Authorization: `Bearer ${token}` }
        });

        if (response.ok) {
          const userData: User = await response.json();
          setUser(userData);
        } else {
          localStorage.removeItem('token');
        }
      } catch {
        localStorage.removeItem('token');
      } finally {
        setIsLoading(false);
      }
    };

    checkAuth();
  }, []);

  const login = async (email: string, password: string): Promise<void> => {
    setError(null);
    setIsLoading(true);

    try {
```

```
      const response = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password })
      });

      if (!response.ok) {
        throw new Error('Invalid credentials');
      }

      const { user: userData, token } = await response.json();
      localStorage.setItem('token', token);
      setUser(userData);
    } catch (err) {
      setError((err as Error).message);
      throw err;
    } finally {
      setIsLoading(false);
    }
  };

  const logout = (): void => {
    localStorage.removeItem('token');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{
      user,
      isAuthenticated: !!user,
      isLoading,
      login,
      logout,
      error
    }}>
      {children}
    </AuthContext.Provider>
  );
}

// Custom hook with safety check
export function useAuth(): AuthContextType {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
```

```
  return context;
}
```

```
// Usage in components

// App.tsx
function App() {
  return (
    <AuthProvider>
      <AppRoutes />
    </AuthProvider>
  );
}

// components/Header.tsx
function Header() {
  const { user, isAuthenticated, logout } = useAuth();

  return (
    <header>
      <h1>My App</h1>
      {isAuthenticated ? (
        <div>
          <span>Welcome, {user?.name}</span>
          <button onClick={logout}>Logout</button>
        </div>
      ) : (
        <a href="/login">Login</a>
      )}
    </header>
  );
}
```

Context API Limitations

Limitation	Description
Performance	ALL consumers re-render when context value changes
No selector	Cannot subscribe to only part of the context
Single responsibility	Should not hold too much unrelated data
Not for high-frequency updates	Every keystroke re-renders all consumers

When to Use Context vs State Management Libraries

Scenario	Recommendation
Theme, locale, auth user	Context API (changes rarely)
Simple shared state (< 5 components)	Context API
Complex state with many actions	Redux / Zustand
High-frequency updates	Zustand / Jotai
Server data caching	React Query / SWR

Part 7: Advanced State Management

7.1 Redux

What Is Redux?

Redux is a predictable state container for JavaScript applications. It centralizes the application state into a single store and enforces unidirectional data flow through actions and reducers.

Why Redux Exists

While Context API works for simple global state, large applications need:

- Predictable state transitions
- Time-travel debugging
- Middleware for side effects
- DevTools for state inspection
- State persistence and hydration

Redux Core Principles

```
+-------------------------------------------------------------+
|                   REDUX CORE PRINCIPLES                      |
+-------------------------------------------------------------+
|                                                             |
|  1. SINGLE SOURCE OF TRUTH                                  |
|     The entire application state is stored in one store     |
|                                                             |
|  2. STATE IS READ-ONLY                                      |
|     The only way to change state is by dispatching actions  |
|                                                             |
|  3. CHANGES BY PURE FUNCTIONS                               |
|     Reducers are pure functions: (state, action) => newState |
+-------------------------------------------------------------+
```

Redux Architecture

```
+-------------------------------------------------------------+
|                    REDUX DATA FLOW                           |
+-------------------------------------------------------------+
```

```
|                                                              |
|  UI Component                                                |
|       |                                                      |
|     dispatches                                               |
|       |                                                      |
|       v                                                      |
|  ACTION  { type: 'ADD_TODO', payload: 'Learn Redux' }       |
|       |                                                      |
|       v                                                      |
|  MIDDLEWARE (optional: thunk, saga, logging)                 |
|       |                                                      |
|       v                                                      |
|  REDUCER  (state, action) => newState                        |
|       |                                                      |
|       v                                                      |
|  STORE  (holds the new state)                                |
|       |                                                      |
|     notifies                                                 |
|       |                                                      |
|       v                                                      |
|  UI Component (re-renders with new state)                    |
+--------------------------------------------------------------+
```

Redux Toolkit Example (Modern Redux)

Redux Toolkit (RTK) is the official, recommended way to write Redux logic. It simplifies store setup, reducer creation, and includes utilities like `createSlice` and `createAsyncThunk`.

```
// store/slices/todosSlice.ts
import { createSlice, createAsyncThunk, PayloadAction } from
'@reduxjs/toolkit';

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

interface TodosState {
  items: Todo[];
  status: 'idle' | 'loading' | 'succeeded' | 'failed';
  error: string | null;
}

const initialState: TodosState = {
```

```
  items: [],
  status: 'idle',
  error: null
};

// Async thunk for API calls
export const fetchTodos = createAsyncThunk('todos/fetchTodos', async () => {
  const response = await fetch('/api/todos');
  const data: Todo[] = await response.json();
  return data;
});

// Create slice (combines actions + reducer)
const todosSlice = createSlice({
  name: 'todos',
  initialState,
  reducers: {
    addTodo: (state, action: PayloadAction<string>) => {
      // RTK uses Immer internally, so "mutation" is safe
      state.items.push({
        id: Date.now(),
        text: action.payload,
        completed: false
      });
    },
    toggleTodo: (state, action: PayloadAction<number>) => {
      const todo = state.items.find(t => t.id === action.payload);
      if (todo) {
        todo.completed = !todo.completed;
      }
    },
    removeTodo: (state, action: PayloadAction<number>) => {
      state.items = state.items.filter(t => t.id !== action.payload);
    }
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchTodos.pending, (state) => {
        state.status = 'loading';
      })
      .addCase(fetchTodos.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.items = action.payload;
      })
      .addCase(fetchTodos.rejected, (state, action) => {
        state.status = 'failed';
        state.error = action.error.message || 'Failed to fetch';
      });
```

```
  }
});

export const { addTodo, toggleTodo, removeTodo } = todosSlice.actions;
export default todosSlice.reducer;
```

```
// store/store.ts
import { configureStore } from '@reduxjs/toolkit';
import todosReducer from './slices/todosSlice';

export const store = configureStore({
  reducer: {
    todos: todosReducer,
    // Add more slices here
  }
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

```
// store/hooks.ts -- Typed hooks
import { useDispatch, useSelector, TypedUseSelectorHook } from 'react-redux';
import type { RootState, AppDispatch } from './store';

export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;
```

```
// Component usage
import { useAppDispatch, useAppSelector } from '../store/hooks';
import { addTodo, toggleTodo, removeTodo, fetchTodos } from
'../store/slices/todosSlice';

function TodoList() {
  const dispatch = useAppDispatch();
  const { items: todos, status, error } = useAppSelector(state => state.todos);
  const [input, setInput] = useState('');

  useEffect(() => {
    if (status === 'idle') {
      dispatch(fetchTodos());
    }
```

```
  }, [status, dispatch]);

  const handleAdd = (): void => {
    if (input.trim()) {
      dispatch(addTodo(input));
      setInput('');
    }
  };

  if (status === 'loading') return <div>Loading...</div>;
  if (status === 'failed') return <div>Error: {error}</div>;

  return (
    <div>
      <input value={input} onChange={e => setInput(e.target.value)} />
      <button onClick={handleAdd}>Add Todo</button>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            <span
              onClick={() => dispatch(toggleTodo(todo.id))}
              style={{ textDecoration: todo.completed ? 'line-through' : 'none'
}}
            >
              {todo.text}
            </span>
            <button onClick={() => dispatch(removeTodo(todo.id))}>X</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

7.2 Zustand

What Is Zustand?

Zustand is a small, fast, and scalable state management library. It provides a simpler API than Redux with less boilerplate while still supporting middleware, persistence, and devtools.

Why Use Zustand?

Feature	Redux Toolkit	Zustand

Bundle Size	~11kb	~1kb
Boilerplate	Moderate (slices, store, hooks)	Minimal (single create call)
Learning Curve	Steeper	Very gentle
DevTools	Yes	Yes (middleware)
Middleware	Yes	Yes
TypeScript	Good	Excellent
Provider Required	Yes	No
Selector Support	Yes	Yes (built-in)

Zustand Store Architecture

```
+-------------------------------------------------------------+
|                    ZUSTAND ARCHITECTURE                      |
+-------------------------------------------------------------+
|                                                             |
|  create() --> STORE (state + actions in one place)          |
|                  |                                          |
|                  |--- state values                          |
|                  |--- action functions                      |
|                  |--- computed values                       |
|                  |                                          |
|                  v                                          |
|  Components subscribe with useStore()                       |
|  Only re-render when SELECTED state changes                 |
|                                                             |
|  No Provider needed!                                        |
|  No dispatch/action creators!                               |
|  No reducers!                                               |
+-------------------------------------------------------------+
```

Zustand Example

```typescript
// store/useCartStore.ts
import { create } from 'zustand';
import { devtools, persist } from 'zustand/middleware';

interface CartItem {
  id: number;
```

```
  name: string;
  price: number;
  quantity: number;
}

interface CartStore {
  items: CartItem[];
  total: number;

  // Actions
  addItem: (item: Omit<CartItem, 'quantity'>) => void;
  removeItem: (id: number) => void;
  updateQuantity: (id: number, quantity: number) => void;
  clearCart: () => void;

  // Computed
  itemCount: () => number;
}

export const useCartStore = create<CartStore>()(
  devtools(
    persist(
      (set, get) => ({
        items: [],
        total: 0,

        addItem: (item) => set((state) => {
          const existing = state.items.find(i => i.id === item.id);
          let newItems: CartItem[];

          if (existing) {
            newItems = state.items.map(i =>
              i.id === item.id ? { ...i, quantity: i.quantity + 1 } : i
            );
          } else {
            newItems = [...state.items, { ...item, quantity: 1 }];
          }

          return {
            items: newItems,
            total: newItems.reduce((sum, i) => sum + i.price * i.quantity, 0)
          };
        }),

        removeItem: (id) => set((state) => {
          const newItems = state.items.filter(i => i.id !== id);
          return {
            items: newItems,
```

```
            total: newItems.reduce((sum, i) => sum + i.price * i.quantity, 0)
          };
        }),

        updateQuantity: (id, quantity) => set((state) => {
          const newItems = state.items.map(i =>
            i.id === id ? { ...i, quantity: Math.max(0, quantity) } : i
          ).filter(i => i.quantity > 0);
          return {
            items: newItems,
            total: newItems.reduce((sum, i) => sum + i.price * i.quantity, 0)
          };
        }),

        clearCart: () => set({ items: [], total: 0 }),

        itemCount: () => get().items.reduce((sum, i) => sum + i.quantity, 0)
      }),
      { name: 'cart-storage' } // localStorage key
    )
  )
);
```

```
// Component usage -- no Provider wrapping needed!

function CartIcon() {
  // Only subscribes to items.length -- won't re-render on other changes
  const itemCount = useCartStore(state => state.items.length);

  return (
    <div className="cart-icon">
      Cart ({itemCount})
    </div>
  );
}

function CartPage() {
  const { items, total, removeItem, updateQuantity, clearCart } =
useCartStore();

  return (
    <div className="cart-page">
      <h2>Shopping Cart</h2>
      {items.map(item => (
        <div key={item.id} className="cart-item">
```

```
          <span>{item.name}</span>
          <span>${item.price}</span>
          <input
            type="number"
            value={item.quantity}
            onChange={e => updateQuantity(item.id, Number(e.target.value))}
          />
          <button onClick={() => removeItem(item.id)}>Remove</button>
        </div>
      ))}
      <div className="total">Total: ${total.toFixed(2)}</div>
      <button onClick={clearCart}>Clear Cart</button>
    </div>
  );
}
```

State Management Decision Tree

```
+-------------------------------------------------------------+
|           STATE MANAGEMENT DECISION TREE                     |
+-------------------------------------------------------------+
|                                                              |
|  Is state used by only ONE component?                        |
|    YES --> useState / useReducer                             |
|    NO  |                                                     |
|        v                                                     |
|  Is it shared by a PARENT and 1-2 CHILDREN?                  |
|    YES --> Lift state up (pass via props)                     |
|    NO  |                                                     |
|        v                                                     |
|  Does state change INFREQUENTLY (theme, auth)?               |
|    YES --> Context API                                        |
|    NO  |                                                     |
|        v                                                     |
|  Is state from a SERVER (API data)?                          |
|    YES --> React Query / SWR                                  |
|    NO  |                                                     |
|        v                                                     |
|  Do you need simple global state with selectors?             |
|    YES --> Zustand                                            |
|    NO  |                                                     |
|        v                                                     |
|  Do you need complex state, middleware, devtools?            |
```

```
|    YES --> Redux Toolkit                                        |
+-----------------------------------------------------------------+
```

Part 8: React with TypeScript

8.1 Why TypeScript in React?

What Is TypeScript?

TypeScript is a statically-typed superset of JavaScript developed by Microsoft. It adds type annotations, interfaces, generics, and compile-time type checking to JavaScript. All valid JavaScript is valid TypeScript, but TypeScript adds an additional layer of safety.

Why Use TypeScript in React Projects?

Benefit	Description
Catch errors at compile time	Type mismatches, missing props, wrong function signatures
Better IDE support	Autocomplete, inline documentation, refactoring
Self-documenting code	Types serve as documentation for components and APIs
Safer refactoring	Rename a prop and TypeScript shows every place it's used
Team scalability	Clear contracts between components, easier onboarding
Reduced runtime bugs	Many categories of bugs become impossible

TypeScript Adoption in Production

```
+-------------------------------------------------------------+
|          WHY PRODUCTION TEAMS CHOOSE TYPESCRIPT              |
+-------------------------------------------------------------+
|                                                              |
|  Without TypeScript (JavaScript):                            |
|  +-------------------------------------------------------+ |
|  | <UserCard name={42} />     // No error until runtime    | |
|  | user.naem                  // Typo undetected           | |
|  | props.onClick()            // Missing prop, crashes     | |
|  +-------------------------------------------------------+ |
|                                                              |
|  With TypeScript:                                            |
|  +-------------------------------------------------------+ |
|  | <UserCard name={42} />     // ERROR: string expected    | |
|  | user.naem                  // ERROR: no 'naem' property | |
```

```
|   | props.onClick()            // ERROR: prop not provided   | |
|   +-------------------------------------------------------+ |
|                                                               |
|   Errors caught BEFORE code reaches production!               |
+---------------------------------------------------------------+
```

8.2 TypeScript Fundamentals for React

Basic Types

```typescript
// Primitive types
let name: string = 'Alice';
let age: number = 28;
let isActive: boolean = true;
let data: null = null;
let value: undefined = undefined;

// Arrays
let names: string[] = ['Alice', 'Bob'];
let ids: Array<number> = [1, 2, 3];

// Tuple
let pair: [string, number] = ['Alice', 28];

// Enum
enum Status {
  Active = 'ACTIVE',
  Inactive = 'INACTIVE',
  Pending = 'PENDING'
}

// Union types
let id: string | number = 'abc';
id = 123; // also valid

// Literal types
let direction: 'up' | 'down' | 'left' | 'right' = 'up';
```

Interfaces vs Types

```typescript
// Interface -- describes object shape, extendable
interface User {
  id: number;
  name: string;
  email: string;
  role: 'admin' | 'user';
  avatar?: string; // optional
}

// Extending interfaces
interface AdminUser extends User {
  permissions: string[];
  lastLogin: Date;
}

// Type -- more flexible, can describe any type
type ID = string | number;
type Status = 'active' | 'inactive' | 'pending';
type UserOrNull = User | null;

// Type for function signatures
type EventHandler = (event: React.MouseEvent) => void;

// Type for complex objects
type ApiResponse<T> = {
  data: T;
  status: number;
  message: string;
  timestamp: Date;
};
```

Feature	Interface	Type
Object shape	Yes	Yes
Extend/Inherit	`extends` keyword	Intersection (`&`)
Union types	No	Yes (`string | number`)
Declaration merging	Yes (auto-merge same name)	No
Primitives/Tuples/Unions	No	Yes
Computed properties	No	Yes
Best for	Object shapes, class contracts	Any type definition

Generics

```
// Generic function -- works with any type
function getFirst<T>(arr: T[]): T | undefined {
  return arr[0];
}

const firstNum = getFirst<number>([1, 2, 3]); // number
const firstStr = getFirst(['a', 'b']); // string (inferred)

// Generic interface
interface ApiResponse<T> {
  data: T;
  loading: boolean;
  error: string | null;
}

// Usage
const userResponse: ApiResponse<User> = {
  data: { id: 1, name: 'Alice', email: 'alice@example.com', role: 'admin' },
  loading: false,
  error: null
};

// Generic with constraints
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const userName = getProperty({ name: 'Alice', age: 28 }, 'name'); // string
```

8.3 Typing React Components

Props Typing

```
// Method 1: Inline type annotation
function Greeting({ name, age }: { name: string; age: number }) {
  return <h1>Hello, {name}! Age: {age}</h1>;
}

// Method 2: Separate interface (recommended for complex props)
```

```
interface ProductCardProps {
  id: number;
  name: string;
  price: number;
  image: string;
  description?: string;
  onAddToCart: (id: number) => void;
  variant?: 'compact' | 'detailed';
}

function ProductCard({
  id,
  name,
  price,
  image,
  description,
  onAddToCart,
  variant = 'compact'
}: ProductCardProps) {
  return (
    <div className={`product-card ${variant}`}>
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p className="price">${price.toFixed(2)}</p>
      {variant === 'detailed' && description && <p>{description}</p>}
      <button onClick={() => onAddToCart(id)}>Add to Cart</button>
    </div>
  );
}
```

Children Props

```
// Typing children
interface LayoutProps {
  children: React.ReactNode; // Most flexible -- accepts anything renderable
}

interface CardProps {
  title: string;
  children: React.ReactElement; // Only accepts JSX elements
}

interface RenderProps {
  children: (data: User) => React.ReactNode; // Render prop pattern
}
```

```
// Example with children
function PageLayout({ children }: LayoutProps) {
  return (
    <div className="page-layout">
      <Header />
      <main>{children}</main>
      <Footer />
    </div>
  );
}
```

Event Handler Types

```
interface FormProps {
  onSubmit: (data: FormData) => void;
}

function ContactForm({ onSubmit }: FormProps) {
  const handleSubmit = (e: React.FormEvent<HTMLFormElement>): void => {
    e.preventDefault();
    // ...
  };

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>): void => {
    console.log(e.target.value);
  };

  const handleClick = (e: React.MouseEvent<HTMLButtonElement>): void => {
    console.log('clicked');
  };

  const handleKeyDown = (e: React.KeyboardEvent<HTMLInputElement>): void => {
    if (e.key === 'Enter') {
      // ...
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={handleChange} onKeyDown={handleKeyDown} />
      <button onClick={handleClick}>Submit</button>
    </form>
```

```
  );
}
```

Common React Event Types

Event	Type
onChange (input)	`React.ChangeEvent<HTMLInputElement>`
onChange (select)	`React.ChangeEvent<HTMLSelectElement>`
onChange (textarea)	`React.ChangeEvent<HTMLTextAreaElement>`
onClick	`React.MouseEvent<HTMLButtonElement>`
onSubmit	`React.FormEvent<HTMLFormElement>`
onKeyDown	`React.KeyboardEvent<HTMLInputElement>`
onFocus	`React.FocusEvent<HTMLInputElement>`
onDrag	`React.DragEvent<HTMLDivElement>`

8.4 Typing Hooks

useState with TypeScript

```
// Simple types -- inferred automatically
const [count, setCount] = useState(0); // number
const [name, setName] = useState(''); // string

// Complex types -- explicit annotation needed
const [user, setUser] = useState<User | null>(null);

interface FormState {
  name: string;
  email: string;
  age: number;
  errors: Record<string, string>;
}

const [form, setForm] = useState<FormState>({
  name: '',
  email: '',
```

```
  age: 0,
  errors: {}
});
```

useReducer with TypeScript

```
// Define state and action types
interface CounterState {
  count: number;
  min: number;
  max: number;
}

type CounterAction =
  | { type: 'INCREMENT' }
  | { type: 'DECREMENT' }
  | { type: 'SET'; payload: number }
  | { type: 'RESET' };

function counterReducer(state: CounterState, action: CounterAction):
CounterState {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, count: Math.min(state.count + 1, state.max) };
    case 'DECREMENT':
      return { ...state, count: Math.max(state.count - 1, state.min) };
    case 'SET':
      return { ...state, count: Math.max(state.min, Math.min(action.payload,
state.max)) };
    case 'RESET':
      return { ...state, count: 0 };
    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(counterReducer, {
    count: 0,
    min: -10,
    max: 10
  });

  return (
    <div>
```

```
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'DECREMENT' })}>-</button>
      <button onClick={() => dispatch({ type: 'INCREMENT' })}>+</button>
      <button onClick={() => dispatch({ type: 'RESET' })}>Reset</button>
    </div>
  );
}
```

useRef with TypeScript

```
// DOM element reference -- initialized with null
const inputRef = useRef<HTMLInputElement>(null);
const divRef = useRef<HTMLDivElement>(null);
const canvasRef = useRef<HTMLCanvasElement>(null);

// Mutable value reference
const timerRef = useRef<NodeJS.Timeout | null>(null);
const countRef = useRef<number>(0);
```

useContext with TypeScript

```
interface NotificationContextType {
  notifications: Notification[];
  addNotification: (message: string, type: 'success' | 'error' | 'info') =>
void;
  removeNotification: (id: string) => void;
  clearAll: () => void;
}

const NotificationContext = createContext<NotificationContextType | undefined>
(undefined);

function useNotifications(): NotificationContextType {
  const context = useContext(NotificationContext);
  if (!context) {
    throw new Error('useNotifications must be used within
NotificationProvider');
  }
  return context;
}
```

8.5 Generics in React Components

Generic Component Pattern

Generics allow you to create components that work with any data type while maintaining type safety.

```
// Generic list component
interface ListProps<T> {
  items: T[];
  renderItem: (item: T, index: number) => React.ReactNode;
  keyExtractor: (item: T) => string | number;
  emptyMessage?: string;
}

function List<T>({
  items,
  renderItem,
  keyExtractor,
  emptyMessage = 'No items found'
}: ListProps<T>) {
  if (items.length === 0) {
    return <p className="empty">{emptyMessage}</p>;
  }

  return (
    <ul className="list">
      {items.map((item, index) => (
        <li key={keyExtractor(item)}>{renderItem(item, index)}</li>
      ))}
    </ul>
  );
}

// Usage with different types
interface User { id: number; name: string }
interface Product { sku: string; title: string; price: number }

// User list
<List<User>
  items={users}
  keyExtractor={u => u.id}
  renderItem={u => <span>{u.name}</span>}
/>

// Product list
<List<Product>
```

```
  items={products}
  keyExtractor={p => p.sku}
  renderItem={p => <span>{p.title} - ${p.price}</span>}
/>
```

Generic Select Component

```
interface SelectProps<T> {
  options: T[];
  value: T | null;
  onChange: (value: T) => void;
  getLabel: (option: T) => string;
  getValue: (option: T) => string | number;
  placeholder?: string;
}

function Select<T>({
  options,
  value,
  onChange,
  getLabel,
  getValue,
  placeholder = 'Select an option'
}: SelectProps<T>) {
  return (
    <select
      value={value ? String(getValue(value)) : ''}
      onChange={(e) => {
        const selected = options.find(o => String(getValue(o)) ===
e.target.value);
        if (selected) onChange(selected);
      }}
    >
      <option value="" disabled>{placeholder}</option>
      {options.map(option => (
        <option key={getValue(option)} value={getValue(option)}>
          {getLabel(option)}
        </option>
      ))}
    </select>
  );
}

// Usage
<Select<User>
```

```
  options={users}
  value={selectedUser}
  onChange={setSelectedUser}
  getLabel={u => u.name}
  getValue={u => u.id}
  placeholder="Choose a user"
/>
```

8.6 Converting React JS to React TS

Step-by-Step Conversion

JavaScript Version:

```
import React, { useState, useEffect } from 'react';

function UserList({ role, onSelect }) {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [search, setSearch] = useState('');

  useEffect(() => {
    fetch(`/api/users?role=${role}`)
      .then(res => res.json())
      .then(data => {
        setUsers(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
      });
  }, [role]);

  const filteredUsers = users.filter(user =>
    user.name.toLowerCase().includes(search.toLowerCase())
  );

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return (
    <div>
```

```
      <input
        value={search}
        onChange={e => setSearch(e.target.value)}
        placeholder="Search users..."
      />
      <ul>
        {filteredUsers.map(user => (
          <li key={user.id} onClick={() => onSelect(user)}>
            {user.name} ({user.email})
          </li>
        ))}
      </ul>
    </div>
  );
}

export default UserList;
```

TypeScript Version:

```
import React, { useState, useEffect } from 'react';

// Step 1: Define data types
interface User {
  id: number;
  name: string;
  email: string;
  role: 'admin' | 'editor' | 'viewer';
  avatar?: string;
}

// Step 2: Define props interface
interface UserListProps {
  role: User['role'];
  onSelect: (user: User) => void;
}

// Step 3: Type the component and all internal state
function UserList({ role, onSelect }: UserListProps): React.ReactElement {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);
  const [search, setSearch] = useState<string>('');

  useEffect(() => {
    const controller = new AbortController();
```

```
    async function fetchUsers(): Promise<void> {
      try {
        const res = await fetch(`/api/users?role=${role}`, {
          signal: controller.signal
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data: User[] = await res.json();
        setUsers(data);
      } catch (err) {
        if ((err as Error).name !== 'AbortError') {
          setError((err as Error).message);
        }
      } finally {
        setLoading(false);
      }
    }

    fetchUsers();
    return () => controller.abort();
  }, [role]);

  // Step 4: Utility functions are also typed
  const filteredUsers: User[] = users.filter((user: User) =>
    user.name.toLowerCase().includes(search.toLowerCase())
  );

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;

  return (
    <div>
      <input
        value={search}
        onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
setSearch(e.target.value)}
        placeholder="Search users..."
      />
      <ul>
        {filteredUsers.map((user: User) => (
          <li key={user.id} onClick={() => onSelect(user)}>
            {user.name} ({user.email})
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```
export default UserList;
```

Conversion Checklist

Step	Action
1	Rename `.jsx` files to `.tsx` (or `.js` to `.ts` for non-JSX)
2	Define interfaces for all data models (`User`, `Product`, etc.)
3	Define interface for component props
4	Add type annotations to `useState` (especially `null` unions)
5	Type event handlers (`React.ChangeEvent`, `React.FormEvent`, etc.)
6	Type `useRef` with the correct HTML element type
7	Add return type annotations to functions
8	Replace `any` with proper types (never use `any` in production)
9	Configure `tsconfig.json` with strict mode

Part 9: React Routing

9.1 Introduction to Client-Side Routing

What Is Client-Side Routing?

In a traditional Multi-Page Application (MPA), navigating to a new URL causes the browser to request an entire new HTML document from the server.

In a React Single-Page Application (SPA), the browser only loads HTML once natively. **Client-side routing** intercepts URL changes (using the HTML5 History API), prevents the browser reload, and simply maps the new URL to a specific React component that renders the new "page" instantly.

Why Use Client-Side Routing?

- **Instant Navigation**: No full-page reloads, resulting in an app-like feel.
- **State Preservation**: The application state (like a shopping cart or user session) is preserved during navigation.
- **Dynamic Routing**: Easy to handle parameters (e.g., `/user/:id`) and render specific data.
- **Lazy Loading**: Route-based code splitting loads components only when their route is accessed.

React Router Architecture

React Router (v6+) is the industry standard for routing in React applications.

```
+-------------------------------------------------------------+
|                    REACT ROUTER V6 FLOW                      |
+-------------------------------------------------------------+
|                                                             |
|  URL Changes (e.g., /products/123)                          |
|       |                                                     |
|       v                                                     |
|  <BrowserRouter> intercepts the change                      |
|       |                                                     |
|       v                                                     |
|  Matches route definition: path="/products/:id"             |
|       |                                                     |
|       v                                                     |
|  Extracts params: { id: "123" }                             |
|       |                                                     |
|       v                                                     |
```

```
|  Renders mapped component: <ProductDetail />                |
+-------------------------------------------------------------+
```

9.2 Route Configuration

Modern React Router uses a configuration-based approach with `createBrowserRouter`.

Basic Routing Setup (TypeScript)

```
// router.tsx
import { createBrowserRouter, RouterProvider } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import NotFound from './pages/NotFound';

// Define the route configuration
const router = createBrowserRouter([
  {
    path: '/',
    element: <Home />, // Rendered at "/"
    errorElement: <NotFound /> // Renders if matching fails or an error is
thrown
  },
  {
    path: '/about',
    element: <About />
  }
]);

// App.tsx
function App() {
  return <RouterProvider router={router} />;
}
```

Navigating Between Routes

Always use the `<Link>` or `<NavLink>` components, never `<a>` tags. An `<a>` tag causes a full page reload.

```
import { Link, NavLink, useNavigate } from 'react-router-dom';

function Navigation() {
  const navigate = useNavigate();

  return (
    <nav>
      {/* Basic navigation */}
      <Link to="/">Home</Link>

      {/* NavLink automatically adds an "active" class when the route matches
*/}
      <NavLink
        to="/about"
        className={({ isActive }) => (isActive ? 'active-link' : '')}
      >
        About
      </NavLink>

      {/* Programmatic navigation */}
      <button onClick={() => navigate('/products')}>
        Go to Products
      </button>

      {/* Navigate back */}
      <button onClick={() => navigate(-1)}>Back</button>
    </nav>
  );
}
```

9.3 Nested Routes

What Are Nested Routes?

Nested routes allow you to render child components within a parent component based on a nested URL path. This is extremely useful for layouts (e.g., a dashboard with a sidebar and a changing main content area).

Implementation

1. Define children in the route config.
2. Use `<Outlet />` in the parent component where the child component should render.

```
// router.tsx
const router = createBrowserRouter([
  {
    path: '/',
    element: <MainLayout />,      // Parent component
    children: [
      {
        index: true,              // Default child when URL is "/"
        element: <DashboardHome />
      },
      {
        path: 'settings',         // URL will be "/settings"
        element: <SettingsPage />
      },
      {
        path: 'profile',          // URL will be "/profile"
        element: <ProfilePage />
      }
    ]
  }
]);

// MainLayout.tsx
import { Outlet, Link } from 'react-router-dom';

function MainLayout() {
  return (
    <div className="layout">
      <aside className="sidebar">
        <nav>
          <Link to="/">Dashboard</Link>
          <Link to="/profile">Profile</Link>
          <Link to="/settings">Settings</Link>
        </nav>
      </aside>
      <main className="content">
        {/* The child component (DashboardHome, SettingsPage, or ProfilePage)
renders here! */}
        <Outlet />
      </main>
    </div>
  );
}
```

9.4 Dynamic Routes and URL Parameters

Implementing Dynamic Routes

Often, URLs carry data, such as an ID to fetch specific details. You define these using a colon (e.g., `:id`).

```
// router.tsx
const router = createBrowserRouter([
  {
    path: '/products',
    element: <ProductList />
  },
  {
    path: '/products/:productId', // Dynamic parameter named "productId"
    element: <ProductDetail />
  }
]);

// ProductDetail.tsx
import { useParams } from 'react-router-dom';
import { useEffect, useState } from 'react';

function ProductDetail() {
  // Extract the parameter matching the route definition
  // Extracting carefully using TS type
  const { productId } = useParams<{ productId: string }>();
  const [product, setProduct] = useState<Product | null>(null);

  useEffect(() => {
    // productId is typed as string | undefined
    if (productId) {
      fetch(`/api/products/${productId}`)
        .then(res => res.json())
        .then(data => setProduct(data));
    }
  }, [productId]);

  if (!product) return <div>Loading...</div>;
  return <h1>{product.name}</h1>;
}
```

Query Parameters

Data appended after a `?` in the URL (e.g., `/search?q=react&sort=asc`).

```
import { useSearchParams } from 'react-router-dom';

function SearchResults() {
  // Returns state value and generic setter
  const [searchParams, setSearchParams] = useSearchParams();

  const query = searchParams.get('q');
  const sort = searchParams.get('sort') || 'desc';

  const updateSort = (newSort: string) => {
    // Update the URL without losing existing parameters
    setSearchParams(prev => {
      prev.set('sort', newSort);
      return prev;
    });
  };

  return (
    <div>
      <h1>Search Results for: {query}</h1>
      <button onClick={() => updateSort('asc')}>Sort Ascending</button>
    </div>
  );
}
```

9.5 Protected Routes

What Is a Protected Route?

A route that should only be accessible if certain conditions are met (e.g., the user is logged in, or the user is an admin). If the condition fails, the user is redirected.

Implementing a Protected Route Wrapper

A standard pattern is a wrapper component that checks authentication state using context.

```
// ProtectedRoute.tsx
import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

interface ProtectedRouteProps {
  requiredRole?: 'admin' | 'user';
```

```
}

function ProtectedRoute({ requiredRole }: ProtectedRouteProps) {
  const { isAuthenticated, user, isLoading } = useAuth();
  const location = useLocation();

  if (isLoading) {
    return <div>Verifying session...</div>;
  }

  // Not logged in -> Redirect to login page
  // Save the intended destination to return them there after login
  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  // Logged in, but checks for specific role
  if (requiredRole && user?.role !== requiredRole) {
    // Return unauthorized view, or redirect to home
    return <Navigate to="/unauthorized" replace />;
  }

  // Authorized -> Render child routes
  return <Outlet />;
}

// router.tsx usage
const router = createBrowserRouter([
  { path: '/login', element: <LoginPage /> },
  {
    // Wrap protected area
    element: <ProtectedRoute />,
    children: [
      { path: '/dashboard', element: <Dashboard /> },
      { path: '/profile', element: <Profile /> },
    ]
  },
  {
    // Wrap admin area
    element: <ProtectedRoute requiredRole="admin" />,
    children: [
      { path: '/admin/users', element: <UserManagement /> }
    ]
  }
]);
```

Part 10: API Integration

10.1 Communicating with Backend APIs

React components need data to render. They get that data by communicating with backend services via HTTP requests (typically REST or GraphQL).

The Fetching Lifecycle

```
+-------------------------------------------------------------+
|                     API INTEGRATION LIFECYCLE                |
+-------------------------------------------------------------+
|                                                              |
|  1. Initialization                                           |
|     Component Mounts -> Sets Loading State: TRUE            |
|                                                              |
|  2. Execution                                                |
|     Sends HTTP Request (fetch/axios)                        |
|                                                              |
|  3. Resolution                                               |
|     +--> SUCCESS: Set Data State, Set Loading: FALSE         |
|     +--> FAILURE: Set Error State, Set Loading: FALSE        |
|                                                              |
|  4. Reactivity                                               |
|     Component re-renders to display Data or Error            |
+-------------------------------------------------------------+
```

10.2 Native Fetch vs Axios

Native `fetch`

Built into modern browsers. Returns a Promise. Note: `fetch` only rejects a promise on network failures, NOT on HTTP errors (like 404 or 500).

```
// fetch pattern
fetch('/api/data')
  // Must manually check response.ok and parse JSON
  .then(res => {
```

```
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return res.json();
  })
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Axios

A popular third-party library that simplifies HTTP requests.

```
// axios pattern
import axios from 'axios';

axios.get('/api/data')
  // Throws automatically on 4xx/5xx, auto-parses JSON
  .then(res => console.log(res.data))
  .catch(err => console.error(err));
```

Axios Instance Configuration (Enterprise Pattern)

In production, you should create a centralized axios instance with interceptors for auth tokens and error handling.

```
// services/api.ts
import axios from 'axios';

// Define the shape of typical API responses
export interface ApiResponse<T> {
  data: T;
  message: string;
}

const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'https://api.example.com/v1',
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json',
  }
});

// Request Interceptor (attaches token)
api.interceptors.request.use(
  (config) => {
```

```
    const token = localStorage.getItem('token');
    if (token && config.headers) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response Interceptor (handles global errors)
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Token expired -> redirect to login or clear session
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```

10.3 Core API Patterns in React

1. The Standard `useEffect` Pattern

This is the baseline way to fetch data inside a component.

```
import { useState, useEffect } from 'react';
import api from '../services/api';

// 1. Define specific interfaces for your data
interface User {
  id: number;
  name: string;
  email: string;
}

function UserList() {
  // 2. Define the exact state variables needed
  const [users, setUsers] = useState<User[]>([]);
```

```
  const [isLoading, setIsLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // 3. Optional but recommended: AbortController for cleanup
    const controller = new AbortController();

    const fetchUsers = async () => {
      try {
        // Reset states
        setIsLoading(true);
        setError(null);

        // API call
        const response = await api.get<User[]>('/users', {
          signal: controller.signal
        });

        // 4. Data transformation & storage
        setUsers(response.data);
      } catch (err: any) {
        // Only set error if it wasn't an intentional abort
        if (err.name !== 'CanceledError') {
          setError(err.response?.data?.message || err.message || 'An error
occurred');
        }
      } finally {
        setIsLoading(false);
      }
    };

    fetchUsers();

    // 5. Cleanup function
    return () => controller.abort();
  }, []); // Run once on mount

  // 6. Handle loading and error UI states
  if (isLoading) return <div>Loading users...</div>;
  if (error) return <div className="error-alert">{error}</div>;

  // 7. Render data
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name} - {user.email}</li>
      ))}
    </ul>
```

```
  );
}
```

2. Service Layer Pattern

Don't write raw `fetch` or `axios` calls in components. Extract them into a service layer.

```
// services/userService.ts
import api from './api';
import { User, CreateUserDto } from '../types/user';

export const userService = {
  // Get all users
  getAll: async (): Promise<User[]> => {
    const { data } = await api.get<User[]>('/users');
    return data;
  },

  // Get by ID
  getById: async (id: number): Promise<User> => {
    const { data } = await api.get<User>(`/users/${id}`);
    return data;
  },

  // Create
  create: async (userData: CreateUserDto): Promise<User> => {
    const { data } = await api.post<User>('/users', userData);
    return data;
  }
};
```

```
// Component using the service
useEffect(() => {
  userService.getAll()
    .then(setUsers)
    .catch(err => setError(err.message))
    .finally(() => setIsLoading(false));
}, []);
```

3. Modern Data Fetching Libraries (React Query)

In modern enterprise applications, standard `useEffect` fetching is replaced by tools like **React Query** or **SWR**, which automatically handle caching, deduplication, background updates, and state management.

```
// Using React Query (TanStack Query)
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { userService } from '../services/userService';

function UserList() {
  const queryClient = useQueryClient();

  // 1. Data fetching is simplified to one hook call
  // Automatically handles loading state, error state, and caching!
  const {
    data: users,
    isLoading,
    isError,
    error
  } = useQuery({
    queryKey: ['users'], // Cache key
    queryFn: userService.getAll
  });

  // 2. Mutations (POST/PUT/DELETE) are also simplified
  const deleteMutation = useMutation({
    mutationFn: userService.delete,
    // When successful, invalidate the 'users' cache to trigger an auto-refetch
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['users'] });
    }
  });

  if (isLoading) return <div>Loading...</div>;
  if (isError) return <div>Error: {(error as Error).message}</div>;

  return (
    <div>
      {users?.map(user => (
        <div key={user.id}>
          {user.name}
          <button onClick={() => deleteMutation.mutate(user.id)}>
            {deleteMutation.isPending ? 'Deleting...' : 'Delete'}
          </button>
        </div>
      ))}
    </div>
```

```
  );
}
```

React Query Benefit	Raw useEffect implementation
Code Amount	2 lines code
Auto Caching	Built in
Auto Retry on fail	Built in
Request Deduplication	Built in
Stale-while-revalidate	Built in

Part 11: Performance Optimization

11.1 Understanding React Rendering

Before optimizing, it is crucial to understand *why* React apps become slow.

The Golden Rule of React Rendering: A component re-renders when:

1. Its state changes
2. Its props change
3. Its parent re-renders

Performance issues almost always stem from **unnecessary re-renders** (rendering when the output hasn't actually changed) or **expensive computations** during render.

11.2 React.memo

What It Does

`React.memo` is a Higher-Order Component (HOC) that memoizes (caches) a component. It prevents the component from re-rendering if its props have not changed.

When to Use It

- The component re-renders often with the EXACT SAME props.
- The component produces the same result given the same props (a pure component).
- The component is heavy/complex, making its re-rendering expensive.

When NOT to Use It

- The component's props change frequently (memoization overhead outweighs benefits).
- The component has `children` passed as essentially new elements.
- Wrapping every component (premature optimization).

Example

```
import React, { memo, useState } from 'react';

interface ExpensiveComponentProps {
  title: string;
  data: number[];
```

```
}

// 1. Wrap the component in React.memo
const ExpensiveDataView = memo(function ExpensiveDataView({ title, data }:
ExpensiveComponentProps) {
  console.log('ExpensiveDataView rendered!');

  // Simulate expensive calculation
  const sum = data.reduce((a, b) => a + b, 0);

  return (
    <div className="expensive-view">
      <h3>{title}</h3>
      <p>Total: {sum}</p>
    </div>
  );
});

// Implementation
function Dashboard() {
  const [count, setCount] = useState(0);
  const [data] = useState([1, 2, 3, 4, 5]);

  return (
    <div>
      {/* Changing count causes Dashboard to re-render */}
      <button onClick={() => setCount(c => c + 1)}>Increment Count: {count}
</button>

      {/* Without React.memo, this would re-render every time count changes.
          With React.memo, it skips re-rendering because 'title' and 'data' are
exactly the same. */}
      <ExpensiveDataView title="My Dashboard Data" data={data} />
    </div>
  );
}
```

The Reference Equality Trap

`React.memo` uses shallow comparison. If you pass a literal Object, Array, or Function as a prop, it is recreated as a *new reference* on every parent render, breaking the memoization.

```
function Dashboard() {
  const [count, setCount] = useState(0);
```

```
  // BAD: [1, 2, 3] is a new array reference on every render
  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
      <ExpensiveDataView title="Stats" data={[1, 2, 3]} /> {/* React.memo FAILS
*/}
    </div>
  );
}
```

To fix this, you combine `React.memo` with `useMemo` and `useCallback`.

11.3 useMemo and useCallback

We covered the internals of these hooks in Section 5, but here is how they apply specifically to performance optimization.

useMemo for Computations and Props

Use `useMemo` to prevent re-creating objects/arrays that are passed as props to memoized components, or to cache expensive calculations.

```
import React, { useMemo, useState } from 'react';

function Dashboard() {
  const [count, setCount] = useState(0);
  const [start, setStart] = useState(1);

  // 1. Caching an expensive calculation
  const expensiveResult = useMemo(() => {
    let result = 0;
    for (let i = start; i < 10000000; i++) {
      result += i;
    }
    return result;
  }, [start]); // Only recompute when 'start' changes

  // 2. Caching an object/array prop
  const chartConfig = useMemo(() => ({
    theme: 'dark',
    showLegend: true,
    dataPoints: [start, start + 1, start + 2]
  }), [start]);
```

```
  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
      <p>Result: {expensiveResult}</p>

      {/* ChartMemoized will NOT re-render when count changes
          because chartConfig reference is stable */}
      <ChartMemoized config={chartConfig} />
    </div>
  );
}
```

useCallback for Functions

Use `useCallback` to prevent re-creating function definitions on every render, which is crucial when passing callbacks to memoized child components.

```
import React, { useState, useCallback, memo } from 'react';

// Memoized child component
const ActionButton = memo(({ label, onAction }: { label: string, onAction: ()
=> void }) => {
  console.log(`Rendering ${label}`);
  return <button onClick={onAction}>{label}</button>;
});

function Parent() {
  const [text, setText] = useState('');
  const [count, setCount] = useState(0);

  // BAD: New function created every render. ActionButton always re-renders.
  const badHandleClick = () => setCount((c) => c + 1);

  // GOOD: Function reference remains identical unless dependencies change.
  // ActionButton skips re-render when 'text' changes.
  const goodHandleClick = useCallback(() => {
    setCount((c) => c + 1);
  }, []); // Empty dependencies = function never changes identity

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />

      <ActionButton label="Bad Button" onAction={badHandleClick} />
```

```
      <ActionButton label="Good Button" onAction={goodHandleClick} />
    </div>
  );
}
```

11.4 Code Splitting and Lazy Loading

What Is Code Splitting?

By default, creating a React app (e.g., via CRA or Vite) bundles all your JavaScript into one large file (`bundle.js`). When a user visits your app, they must download the entire bundle before the app starts, even if they only visit the home page and never see the complex Settings or Admin pages.

Code splitting allows you to split your code into smaller chunks, loading them only when they are needed.

React.lazy and Suspense

`React.lazy` lets you render a dynamic import as a regular component. It must be wrapped in a `<Suspense>` component, which displays a fallback UI while the chunk is downloading.

```
// App.tsx
import React, { Suspense } from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

// 1. Regular import (included in main bundle)
import Home from './pages/Home';

// 2. Lazy imports (split into separate JS chunks)
// These files will ONLY download if/when the user navigates to their route
const AdminDashboard = React.lazy(() => import('./pages/AdminDashboard'));
const ReportingCharts = React.lazy(() => import('./pages/ReportingCharts'));

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/admin">Admin</Link>
        <Link to="/reports">Reports</Link>
      </nav>
```

```
      {/* 3. Suspense boundary provides the fallback UI during download */}
      <Suspense fallback={<div className="spinner">Loading route...</div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/admin" element={<AdminDashboard />} />
          <Route path="/reports" element={<ReportingCharts />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

Route-Based vs Component-Based Splitting

- **Route-Based (Most Common)**: Split chunks by page (as shown above).
- **Component-Based**: Lazy load heavy components (like a rich text editor or a 3D charting library) even if they are on the initial route, loading them only when the user clicks a button to open them.

Part 12: Form Handling

Forms are a critical part of almost every web application. React handles forms differently than traditional HTML.

12.1 Controlled vs Uncontrolled Components

Controlled Components

In a controlled component, form data is handled entirely by React state. The state is the "single source of truth".

```
+-------------------------------------------------------------+
|                    CONTROLLED COMPONENT                      |
+-------------------------------------------------------------+
|                                                             |
|  1. Input value is tied strictly to state: value={email}    |
|  2. Keystroke fires onChange event                          |
|  3. onChange handler updates state: setEmail(e.target.value) |
|  4. State change triggers re-render                         |
|  5. Input displays new state value                          |
+-------------------------------------------------------------+
```

```
import React, { useState } from 'react';

function ControlledForm() {
  // 1. Create state
  const [username, setUsername] = useState('');

  // 2. Handle submit
  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    console.log('Submitted:', username);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Username:
        <input
          type="text"
          // 3. Bind value and onChange
```

```
          value={username}
          onChange={(e) => setUsername(e.target.value)}
        />
      </label>
      <button type="submit">Submit</button>

      <p>Live preview: {username}</p>
    </form>
  );
}
```

Uncontrolled Components

In an uncontrolled component, form data is handled by the DOM itself, just like traditional HTML. You use a `useRef` to read the value from the DOM only when needed (usually on submit).

```
import React, { useRef } from 'react';

function UncontrolledForm() {
  // 1. Create a ref
  const emailRef = useRef<HTMLInputElement>(null);

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    // 2. Read value directly from the DOM node on submit
    const emailValue = emailRef.current?.value;
    console.log('Submitted:', emailValue);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Email:
        {/* 3. Attach the ref. Do NOT use standard 'value' or 'onChange' */}
        <input type="text" ref={emailRef} defaultValue="test@example.com" />
      </label>
      <button type="submit">Submit</button>
    </form>
  );
}
```

Which should you use?

Scenario	Controlled	Uncontrolled
Instant validation (as you type)	Yes	No
Conditionally disabling submit button	Yes	No
Enforcing input format (e.g., credit card)	Yes	No
Single, simple form submission	Overkill	Yes
Integrating with non-React libraries	Harder	Easier
File inputs (`<input type="file">`)	No (Read-only)	Must use

Rule of Thumb: Use Controlled Components for 95% of use cases in React.

12.2 Handling Multiple Inputs

When you have many inputs, creating a separate `useState` for each becomes messy. Instead, use a single state object.

```
import React, { useState } from 'react';

interface FormData {
  firstName: string;
  lastName: string;
  email: string;
  role: string;
  agreedToTerms: boolean;
}

function ComplexForm() {
  const [formData, setFormData] = useState<FormData>({
    firstName: '',
    lastName: '',
    email: '',
    role: 'user',
    agreedToTerms: false
  });

  // Generic change handler based on the 'name' attribute
  const handleChange = (e: React.ChangeEvent<HTMLInputElement |
HTMLSelectElement>) => {
    const target = e.target;
    const name = target.name;
```

```
  // Checkboxes use 'checked' instead of 'value'
  const value = target.type === 'checkbox'
    ? (target as HTMLInputElement).checked
    : target.value;

  setFormData(prev => ({
    ...prev,
    [name]: value
  }));
};

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  console.log('Form data to send to API:', formData);
};

return (
  <form onSubmit={handleSubmit}>
    <input
      name="firstName"
      value={formData.firstName}
      onChange={handleChange}
      placeholder="First Name"
    />

    <input
      name="lastName"
      value={formData.lastName}
      onChange={handleChange}
      placeholder="Last Name"
    />

    <select name="role" value={formData.role} onChange={handleChange}>
      <option value="user">User</option>
      <option value="admin">Admin</option>
    </select>

    <label>
      <input
        type="checkbox"
        name="agreedToTerms"
        checked={formData.agreedToTerms}
        onChange={handleChange}
      />
      I agree to the terms
    </label>
```

```
      <button type="submit" disabled={!formData.agreedToTerms}>
        Register
      </button>
    </form>
  );
}
```

12.3 Form Validation

Forms require validation before submission. You can manage validation state manually, or use industry-standard libraries.

Manual Validation Example

```
import React, { useState } from 'react';

function ValidationForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState<{ email?: string; password?: string }>
({});

  const validate = () => {
    const newErrors: { email?: string; password?: string } = {};
    let isValid = true;

    if (!email) {
      newErrors.email = 'Email is required';
      isValid = false;
    } else if (!/\S+@\S+\.\S+/.test(email)) {
      newErrors.email = 'Email is invalid';
      isValid = false;
    }

    if (password.length < 6) {
      newErrors.password = 'Password must be at least 6 characters';
      isValid = false;
    }

    setErrors(newErrors);
    return isValid;
  };
```

```
  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (validate()) {
      console.log('Submitting:', { email, password });
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <div className="form-group">
        <label>Email</label>
        <input
          value={email}
          onChange={(e) => {
            setEmail(e.target.value);
            // Optional: Clear error when user starts typing again
            if (errors.email) setErrors({...errors, email: undefined});
          }}
          className={errors.email ? 'is-invalid' : ''}
        />
        {errors.email && <span className="error-text">{errors.email}</span>}
      </div>

      {/* Password field similar... */}

      <button type="submit">Login</button>
    </form>
  );
}
```

Form Libraries (React Hook Form + Zod)

For production applications with complex forms, managing onChange, error state, and touched state manually is tedious.

React Hook Form is the modern standard for React forms because it:

1. Minimizes re-renders by utilizing uncontrolled inputs internally.
2. Integrates perfectly with **Zod** (or Yup) for schema-based validation.

```
// Example of a Production-Grade Form using React Hook Form + Zod
import React from 'react';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import * as z from 'zod';
```

```
// 1. Define the validation schema using Zod
const registrationSchema = z.object({
  username: z.string().min(3, "Username must be at least 3 characters"),
  email: z.string().email("Invalid email address"),
  password: z.string().min(8, "Password must be at least 8 characters"),
  confirmPassword: z.string()
}).refine((data) => data.password === data.confirmPassword, {
  message: "Passwords don't match",
  path: ["confirmPassword"], // Attach the error to the confirmPassword field
});

// Infer the TypeScript type from the Zod schema
type RegistrationFormInputs = z.infer<typeof registrationSchema>;

function ProductionForm() {
  // 2. Initialize React Hook Form
  const {
    register,           // Registers inputs to the form
    handleSubmit,       // Handles form submission
    formState: { errors, isSubmitting } // Extracts error and loading states
  } = useForm<RegistrationFormInputs>({
    resolver: zodResolver(registrationSchema)
  });

  // 3. Submit handler (only called if validation passes)
  const onSubmit = async (data: RegistrationFormInputs) => {
    try {
      // Simulate API call
      await new Promise(resolve => setTimeout(resolve, 2000));
      console.log("Success! Sending data:", data);
    } catch (error) {
      console.error("Submission failed", error);
    }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="form">

      <div className="field">
        <label>Username</label>
        {/* register('username') maps ref, onChange, onBlur automatically */}
        <input {...register("username")} />
        {errors.username && <span className="error">{errors.username.message}
</span>}
      </div>

      <div className="field">
```

```
        <label>Email</label>
        <input type="email" {...register("email")} />
        {errors.email && <span className="error">{errors.email.message}</span>}
      </div>

      <div className="field">
        <label>Password</label>
        <input type="password" {...register("password")} />
        {errors.password && <span className="error">{errors.password.message}
</span>}
      </div>

      <div className="field">
        <label>Confirm Password</label>
        <input type="password" {...register("confirmPassword")} />
        {errors.confirmPassword && <span className="error">
{errors.confirmPassword.message}</span>}
      </div>

      <button type="submit" disabled={isSubmitting}>
        {isSubmitting ? "Registering..." : "Register"}
      </button>

    </form>
  );
}
```

Why React Hook Form is Better

Approach	Re-renders	Validation Logic	Boilerplate
Manual Controlled	Every keystroke	Manual JS logic	High
React Hook Form	Only on error/submit	Zod Schema	Low

Part 13: Real-World Application Examples

This section provides complete, functioning examples of common real-world features you will build in production React applications.

13.1 User Authentication System

A complete authentication flow using Context API, simulating an API login, storing a token, and managing the user session.

```
// 1. types/auth.ts
export interface User {
  id: string;
  name: string;
  email: string;
  role: 'admin' | 'user';
}

// 2. context/AuthContext.tsx
import React, { createContext, useContext, useState, useEffect, ReactNode }
from 'react';

interface AuthContextType {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  // Check token on mount
  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) {
      // Simulate API verification
      setTimeout(() => {
        setUser({ id: '1', name: 'Alice', email: 'alice@example.com', role:
'admin' });
```

```
        setIsLoading(false);
      }, 500);
    } else {
      setIsLoading(false);
    }
  }, []);

  const login = async (email: string, password: string) => {
    setIsLoading(true);
    return new Promise<void>((resolve, reject) => {
      setTimeout(() => {
        if (email === 'admin@demo.com' && password === 'admin123') {
          localStorage.setItem('token', 'fake-jwt-token-123');
          setUser({ id: '1', name: 'Alice Admin', email, role: 'admin' });
          resolve();
        } else {
          reject(new Error('Invalid credentials'));
        }
        setIsLoading(false);
      }, 1000);
    });
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, isAuthenticated: !!user, isLoading,
login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be inside AuthProvider');
  return context;
};

// 3. components/Login.tsx
import React, { useState } from 'react';

export function Login() {
  const { login, isAuthenticated } = useAuth();
  const [email, setEmail] = useState('');
```

```
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  if (isAuthenticated) {
    return <div>You are already logged in!</div>;
  }

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      await login(email, password);
    } catch (err: any) {
      setError(err.message);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="login-form">
      <h2>Please Sign In</h2>
      {error && <div className="error-alert">{error}</div>}
      <input
        type="email"
        value={email}
        onChange={e => setEmail(e.target.value)}
        placeholder="admin@demo.com"
        required
      />
      <input
        type="password"
        value={password}
        onChange={e => setPassword(e.target.value)}
        placeholder="admin123"
        required
      />
      <button type="submit">Log In</button>
    </form>
  );
}

// 4. components/Dashboard.tsx
import React from 'react';

export function Dashboard() {
  const { user, logout } = useAuth();

  return (
    <div className="dashboard">
      <header>
```

```
        <h1>Welcome, {user?.name}</h1>
        <button onClick={logout}>Sign Out</button>
      </header>
      <div className="content">
        <p>Your role is: <strong>{user?.role}</strong></p>
        {user?.role === 'admin' && (
          <div className="admin-panel">
            <h3>Admin Controls</h3>
            <button>Manage Users</button>
          </div>
        )}
      </div>
    </div>
  );
}
```

13.2 Data Table with Filtering and Sorting

A common enterprise requirement: displaying data, searching, and sorting columns.

```
import React, { useState, useMemo } from 'react';

interface Employee {
  id: number;
  name: string;
  department: string;
  role: string;
  salary: number;
}

const MOCK_DATA: Employee[] = [
  { id: 1, name: 'John Doe', department: 'Engineering', role: 'Frontend Dev',
salary: 90000 },
  { id: 2, name: 'Jane Smith', department: 'Design', role: 'UX UI', salary:
85000 },
  { id: 3, name: 'Sam Johnson', department: 'Engineering', role: 'Backend Dev',
salary: 110000 },
  { id: 4, name: 'Anna Lee', department: 'HR', role: 'Manager', salary: 75000
},
];

type SortKey = keyof Employee;
type SortDirection = 'asc' | 'desc';
```

```
export function EmployeeTable() {
  const [search, setSearch] = useState('');
  const [deptFilter, setDeptFilter] = useState('All');
  const [sortConfig, setSortConfig] = useState<{ key: SortKey; direction:
SortDirection } | null>(null);

  // Derive all options for the select dropdown dynamically
  const departments = ['All', ...Array.from(new Set(MOCK_DATA.map(emp =>
emp.department)))];

  // Expensive filtering and sorting is memoized
  const processedData = useMemo(() => {
    let result = [...MOCK_DATA];

    // 1. Search filter
    if (search) {
      const lowerSearch = search.toLowerCase();
      result = result.filter(emp =>
        emp.name.toLowerCase().includes(lowerSearch) ||
        emp.role.toLowerCase().includes(lowerSearch)
      );
    }

    // 2. Department filter
    if (deptFilter !== 'All') {
      result = result.filter(emp => emp.department === deptFilter);
    }

    // 3. Sorting
    if (sortConfig !== null) {
      result.sort((a, b) => {
        const valA = a[sortConfig.key];
        const valB = b[sortConfig.key];

        if (valA < valB) return sortConfig.direction === 'asc' ? -1 : 1;
        if (valA > valB) return sortConfig.direction === 'asc' ? 1 : -1;
        return 0;
      });
    }

    return result;
  }, [search, deptFilter, sortConfig]);

  const requestSort = (key: SortKey) => {
    let direction: SortDirection = 'asc';
    if (sortConfig && sortConfig.key === key && sortConfig.direction === 'asc')
{
      direction = 'desc';
```

```
    }
    setSortConfig({ key, direction });
  };

  return (
    <div className="table-container">
      <div className="controls">
        <input
          placeholder="Search name or role..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
        />
        <select value={deptFilter} onChange={(e) =>
setDeptFilter(e.target.value)}>
          {departments.map(dept => (
            <option key={dept} value={dept}>{dept}</option>
          ))}
        </select>
        <span>Showing {processedData.length} employees</span>
      </div>

      <table>
        <thead>
          <tr>
            <th onClick={() => requestSort('name')}>Name ↕</th>
            <th onClick={() => requestSort('department')}>Department ↕</th>
            <th onClick={() => requestSort('role')}>Role ↕</th>
            <th onClick={() => requestSort('salary')}>Salary ↕</th>
          </tr>
        </thead>
        <tbody>
          {processedData.length > 0 ? (
            processedData.map(emp => (
              <tr key={emp.id}>
                <td>{emp.name}</td>
                <td>{emp.department}</td>
                <td>{emp.role}</td>
                <td>${emp.salary.toLocaleString()}</td>
              </tr>
            ))
          ) : (
            <tr><td colSpan={4}>No results found</td></tr>
          )}
        </tbody>
      </table>
    </div>
```

```
  );
}
```

Part 14: React JS vs React TypeScript

14.1 Why Choose One Over the Other?

Building a React application is fundamentally different depending on whether you use plain JavaScript (`.jsx`) or TypeScript (`.tsx`).

JavaScript offers speed, flexibility, and a lower barrier to entry. **TypeScript** offers robust architecture, compiler safety, self-documenting APIs, and scale.

Today, **TypeScript is the industry standard** for new React enterprise applications.

14.2 Feature Comparison

Feature	React + JavaScript	React + TypeScript
Type Safety	None (unless using PropTypes)	Strict compile-time checks
Error Detection	Runtime (found in the browser)	Compile-time (found in the IDE)
Refactoring	Dangerous (rely on global search)	Safe (compiler flags missing fields)
Component API	Unclear (must read component implementation)	Explicit (Interfaces define exact props)
Learning Curve	Shorter	Steeper (requires understanding TS concepts)
Setup Time	Instant (CRA, Vite)	Slightly longer (TSConfig, strictness)
Team Size	Best for small teams / solo	Essential for large teams (5+ devs)
Maintainability	Harder linearly as app grows	Easier linearly as app grows
Build Speed	Faster	Slower (type checking phase)

14.3 Side-by-Side Code Examples

1. Defining Component Props

JavaScript (with PropTypes):

```
import React from 'react';
import PropTypes from 'prop-types';

function Button({ label, onClick, disabled, variant }) {
  const className = `btn btn-${variant} ${disabled ? 'disabled' : ''}`;
  return (
    <button className={className} onClick={onClick} disabled={disabled}>
      {label}
    </button>
  );
}

Button.propTypes = {
  label: PropTypes.string.isRequired,
  onClick: PropTypes.func.isRequired,
  disabled: PropTypes.bool,
  variant: PropTypes.oneOf(['primary', 'secondary', 'danger'])
};

Button.defaultProps = {
  disabled: false,
  variant: 'primary'
};
```

TypeScript:

```
import React from 'react';

// The interface acts as PropTypes AND documentation
interface ButtonProps {
  label: string;
  onClick: () => void;
  disabled?: boolean;
  variant?: 'primary' | 'secondary' | 'danger';
}

// Defaults defined directly in parameters
export function Button({
  label,
  onClick,
  disabled = false,
  variant = 'primary'
}: ButtonProps) {
  const className = `btn btn-${variant} ${disabled ? 'disabled' : ''}`;
```

```
  return (
    <button className={className} onClick={onClick} disabled={disabled}>
      {label}
    </button>
  );
}
```

2. State Formatting & Auto-completion

JavaScript:

```
// JS doesn't know what 'user' contains until assigned
const [user, setUser] = useState(null);

// Later in code... no auto-complete available here
// Might accidentally type user.first_name instead of user.firstName
const name = user ? user.name : 'Guest';
```

TypeScript:

```
interface User {
  id: string;
  name: string;
  email: string;
}

// TS knows user is either null OR a User object
const [user, setUser] = useState<User | null>(null);

// IDE gives auto-complete: user.id, user.name, user.email
// TS throws error if you type user.first_name
const name = user ? user.name : 'Guest';
```

3. API Responses

JavaScript:

```
const fetchProduct = async () => {
  const res = await fetch('/api/product/1');
  const data = await res.json();
  // 'data' is ANY type. You don't know what it contains.
```

```
  console.log(data.pice); // Typo! Undefined at runtime.
};
```

TypeScript:

```
interface ProductResponse {
  id: number;
  title: string;
  price: number;
}

const fetchProduct = async () => {
  const res = await fetch('/api/product/1');
  const data: ProductResponse = await res.json();

  // TS ERROR: Property 'pice' does not exist on type 'ProductResponse'
  // Did you mean 'price'?
  console.log(data.pice);
};
```

Conclusion

While JavaScript is fine for quick prototypes or small, short-lived projects, the **investment in TypeScript pays off exponentially** in any project that will be maintained for more than a few months or worked on by more than one person.

Part 15: Production Architecture

15.1 Scaling React Applications

When building enterprise applications, the simple "components" and "pages" folders are no longer enough. You must think about how state flows, how APIs are integrated, how styles are handled, and how features are decoupled.

A production architecture should enforce:

- **Separation of Concerns:** UI does not mix with API calls or business logic.
- **Modularity:** Features can be easily added, modified, or removed.
- **Predictability:** Developers know exactly where to put new code.

15.2 Feature-Sliced Design (Production Standard)

The modern standard for large React applications is **Feature-Sliced Design**. Instead of grouping by file type (all hooks together, all components together), you group files by **Feature Area**.

Folder Structure

```
src/
|-- app/                    # Application setup (entry point, global providers)
|   |-- store.ts            # Global Redux store
|   |-- router.tsx          # Main React Router setup
|   +-- App.tsx             # Root component
|
|-- shared/                 # Truly generic, agnostic utilities & UI
|   |-- ui/                 # Buttons, Modals, Inputs (no business logic)
|   |-- api/                # Axios instance configuration
|   |-- utils/              # Generic formatters, regex constants
|   +-- hooks/              # useDebounce, useLocalStorage
|
|-- entities/               # Business entities (domain logic)
|   |-- User/
|   |   |-- User.types.ts   # User interfaces
|   |   |-- User.api.ts     # fetchUser endpoints
|   |   +-- UserAvatar.tsx  # Display component for user
|   +-- Product/
|
|-- features/               # User interactions combining entities
```

```
|   |-- AuthUser/           # Login/Logout functionality
|   |   |-- api/
|   |   |-- model/          # Redux slice or Zustand store for Auth
|   |   +-- ui/             # LoginForm.tsx, LogoutButton.tsx
|   +-- AddToCart/
|
|-- widgets/                # Composition of features into independent blocks
|   |-- Header/             # Contains Title + AuthUser profile
|   +-- Sidebar/
|
|-- pages/                  # Route components composing widgets
|   |-- DashboardPage/
|   +-- LoginPage/
```

Why this is powerful:

If you need to delete the "AddToCart" functionality, you delete the `features/AddToCart` folder. You do not have to hunt for its components, its Redux actions, and its API calls across 5 different overarching folders.

15.3 Architecture Diagrams

Component Hierarchy (Atomic Design Principles)

Modern production React UI builds outwards from the smallest pieces.

```
+-------------------------------------------------------------+
|                    UI COMPONENT HIERARCHY                    |
+-------------------------------------------------------------+
|                                                             |
|  ATOMS (Shared UI)                                          |
|  <Button /> | <Input /> | <Typography /> | <Icon />         |
|      No business logic, purely presentational.              |
|                                                             |
|        ^                                                    |
|        |                                                    |
|  MOLECULES (Combined Atoms)                                 |
|  <SearchBox /> (Input + Icon + Button)                      |
|  <FormField /> (Label + Input + ErrorText)                  |
|                                                             |
|        ^                                                    |
|        |                                                    |
|  ORGANISMS (Widgets - State Aware)                          |
|  <TopNavbar /> | <ProductCard /> | <LoginForm />            |
```

```
|  Connects to Redux/Context. Handles local logic.             |
|                                                              |
|        ^                                                     |
|        |                                                     |
|  TEMPLATES (Layouts)                                         |
|  <DashboardLayout />                                         |
|  Defines the grid/flexboxes where content will go.           |
|                                                              |
|        ^                                                     |
|        |                                                     |
|  PAGES (Routes)                                              |
|  <UserProfilePage />                                         |
|  Fetches data (React Query), passes to Templates/Organisms   |
+--------------------------------------------------------------+
```

State Management Flow

Production applications do not use just one state manager. They split state by its purpose.

```
+--------------------------------------------------------------+
|                    STATE CLASSIFICATION                       |
+--------------------------------------------------------------+
|                                                              |
|  1. SERVER STATE (Data from API)                             |
|     Tool: React Query / RTK Query / SWR                      |
|     Responsibility: Caching, loading/error states, retries,  |
|                     background fetching.                     |
|                                                              |
|  2. GLOBAL APP STATE (Client-side only)                      |
|     Tool: Zustand / Redux / Context (Theme)                  |
|     Responsibility: User session, UI Theme, Auth Token,      |
|                     Sidebar collapse state.                  |
|                                                              |
|  3. URL STATE (Sharable state)                               |
|     Tool: React Router (useSearchParams)                     |
|     Responsibility: Search queries, pagination, active tab,  |
|                     sorting filters.                         |
|                                                              |
|  4. LOCAL COMPONENT STATE                                    |
|     Tool: useState / useReducer / React Hook Form            |
|     Responsibility: Form inputs, modal open/close, toggles.  |
+--------------------------------------------------------------+
```

API Integration Layer

The API layer must be abstracted away from the UI components. A component should never know *how* `fetch` or `axios` works. A component should only call a function or a React Query hook.

```
+-------------------------------------------------------------+
|                 API ABSTRACTION LAYER                        |
+-------------------------------------------------------------+
|                                                             |
|  [ COMPONENT LAYER ]                                        |
|  <ProductListPage />                                        |
|          |                                                  |
|          | calls useQuery(getProducts)                      |
|          v                                                  |
|                                                             |
|  [ DATA FETCHER LAYER ] (React Query)                       |
|  useProductsHook.ts                                         |
|  Handles loading state, caches the return value.            |
|          |                                                  |
|          | calls service                                    |
|          v                                                  |
|                                                             |
|  [ SERVICE LAYER ]                                          |
|  productService.ts                                          |
|  const getProducts = () => axiosInstance.get('/products')   |
|          |                                                  |
|          | uses HTTP Client                                 |
|          v                                                  |
|                                                             |
|  [ HTTP CLIENT LAYER ]                                      |
|  axiosConfig.ts                                             |
|  Intercepts requests, appends Bearer tokens.                |
|  Intercepts responses, handles 401 unauth errors globally.  |
|          |                                                  |
|          v                                                  |
|  [ BACKEND API SERVER ]                                     |
+-------------------------------------------------------------+
```

15.4 Environment Configuration

Production apps require management of variables across `development`, `staging`, and `production`.

In React (e.g., using Vite or Create React App), environment variables are strictly separated.

`.env.development`

```
VITE_API_BASE_URL=http://localhost:8080/api/v1
VITE_ENABLE_MOCKING=true
VITE_ENVIRONMENT=dev
```

`.env.production`

```
VITE_API_BASE_URL=https://api.myenterpriseapp.com/v1
VITE_ENABLE_MOCKING=false
VITE_ENVIRONMENT=prod
```

Usage:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL // Vite syntax
  // OR: process.env.REACT_APP_API_BASE_URL // CRA syntax
});
```

15.5 Error Monitoring & Boundaries

A crash in a JavaScript application results in a "White Screen of Death". Production applications use Error Boundaries and monitoring tools (like Sentry).

Global Error Boundary: Catches UI rendering errors.

```
import { ErrorBoundary } from 'react-error-boundary';

function FallbackUI({ error, resetErrorBoundary }) {
  return (
    <div className="error-container">
      <h1>Something went wrong.</h1>
      <pre>{error.message}</pre>
      <button onClick={resetErrorBoundary}>Try again</button>
    </div>
  );
}
```

```
// In App.tsx
function App() {
  return (
    <ErrorBoundary
      FallbackComponent={FallbackUI}
      onError={(error) => {
        // Log to external service like Sentry or Datadog
        Sentry.captureException(error);
      }}
    >
      <RouterProvider router={router} />
    </ErrorBoundary>
  );
}
```

By following Feature-Sliced Design, strict state separation, and implementing API abstraction and boundaries, React applications scale smoothly across dozens of developers and millions of users.

Part 16: Common Interview Questions

This section contains 50 essential React interview questions categorized by difficulty and topic.

Core Concepts

1. What is React? React is an open-source JavaScript library for building user interfaces, mostly used for single-page applications. It focuses on the view layer and handles UI rendering through reusable components.

2. What are the key features of React?

- Virtual DOM for fast performance.
- Component-based architecture.
- Declarative UI.
- JSX syntax.
- Unidirectional data flow.

3. What is JSX? JSX stands for JavaScript XML. It is a syntax extension for JavaScript that allows you to write HTML-like code inside JavaScript files. Browsers cannot read it natively; it must be transpiled by Babel into `React.createElement()` calls.

4. What is the Virtual DOM? The Virtual DOM is a lightweight, in-memory representation of the real DOM. When state changes, React creates a new VDOM, compares it with the previous VDOM (diffing), and calculates the minimal set of changes (reconciliation) needed to update the real DOM.

5. Why can't browsers read JSX? Browsers only understand plain JavaScript objects. JSX must be converted to standard JavaScript objects using tools like Babel before it can be executed.

6. What is the difference between Real DOM and Virtual DOM? Real DOM manipulation is slow, updates the entire tree, and directly affects the HTML document. Virtual DOM manipulation is fast, operates in memory, and calculates minimal diffs before applying changes to the Real DOM.

7. Define "Declarative Programming" in React. In declarative programming, you tell React *what* the UI should look like for a given state, and React figures out *how* to update the DOM correctly. This contrasts with imperative programming, where you write step-by-step instructions.

8. What is Reconciliation? Reconciliation is React's algorithm (currently called Fiber) for comparing two Virtual DOM trees and applying the minimal necessary operations to the real DOM.

9. Why do we need keys in React lists? Keys help React uniquely identify items in a list. During reconciliation, React uses keys to determine which items were added, removed, or reordered, avoiding unnecessary re-renders of the entire list.

10. What happens if you use the array index as a key? Using the index as a key is an anti-pattern if the list order can change (e.g., sorting, adding/removing items). It can cause severe rendering bugs where state gets attached to the wrong list item.

Components & Props

11. What is the difference between Functional and Class components? Class components require extending `React.Component`, have lifecycle methods, and use `this.state`. Functional components are plain JS functions that accept props, return JSX, and manage state using Hooks. Functional components are the modern standard.

12. What are props in React? Props (properties) are read-only objects passed from a parent component to a child component to configure its rendering or behavior.

13. Can a child component modify its own props? No. Props are strictly read-only. Modifying them violates React's unidirectional data flow.

14. What is `children` prop? `children` is a special prop that allows components to pass unconfigured React elements directly inside the opening and closing tags of a component (e.g., `<Card><p>Hello</p></Card>`).

15. What are defaultProps? Default props define fallback values for props if the parent component does not provide them. In modern functional components, default parameters are preferred over the `defaultProps` property.

16. What is Prop Drilling? Prop drilling is the process of passing props through multiple levels of intermediate components that don't need the data, simply to reach a deeply nested child that does.

17. How do you solve Prop Drilling? By using the Context API, state management libraries like Redux or Zustand, or by restructuring the component hierarchy using component composition (passing rendered elements as props).

18. What is a Higher-Order Component (HOC)? An HOC is a function that takes a component and returns a new component, typically to add shared logic or inject props. Example: `withRouter` or `connect` in older Redux.

State & Lifecycle

19. What is state in React? State is a built-in object that stores data or information about the component. Unlike props, state is fully managed by the component itself and can be mutated (via setter functions).

20. Why should you never update state directly? If you mutate state directly (`this.state.count = 2`), React will not be notified of the change, and the component will not re-render. Always use the setter function (e.g., `setCount`).

21. Are state updates synchronous or asynchronous? State updates in React are asynchronous. React batches multiple state updates together into a single re-render for performance optimization.

22. How do you update state based on previous state? You should pass an updater function to the `setState` hook instead of the value directly: `setCount(prevCount => prevCount + 1)` .

23. Name the main lifecycle phases of a component. Mounting (insertion into DOM), Updating (re-rendering due to state/prop changes), and Unmounting (removal from DOM).

24. What is `componentDidMount` equivalent in Hooks? `useEffect(() => { ... }, [])` (with an empty dependency array).

25. What is `componentWillUnmount` equivalent in Hooks? The cleanup function returned inside `useEffect` : `useEffect(() => { return () => { ...cleanup logic... } }, [])` .

Hooks

26. What are Hooks? Hooks are functions introduced in React 16.8 that allow you to use state and lifecycle features inside functional components without writing a class.

27. What are the Rules of Hooks?

1. Only call Hooks at the top level of a functional component.
2. Never call Hooks inside loops, conditions, or nested functions.
3. Only call Hooks from React Function Components or Custom Hooks.

28. Explain `useState` . It is a Hook that lets you add React state to function components. It returns an array with two values: the current state and a function to update it.

29. Explain `useEffect` . It is a Hook used to manage side effects (fetching data, subscribing to events, manual DOM manipulation) in function components. It runs after the render phase.

30. What is the dependency array in `useEffect` ? It is an array of variables that the effect depends on. If any variable in the array changes between renders, the effect re-runs. If omitted, the effect runs on *every* render. Provide an empty array `[]` to run only once on mount.

31. Explain `useContext`. It is a Hook that accepts a context object (created by `React.createContext`) and returns the current context value. It prevents prop drilling.

32. Explain `useReducer`. An alternative to `useState` for managing complex state objects or state transitions that depend on previous state. It follows the Redux pattern: `(state, action) => newState`.

33. What is the difference between `useMemo` and `useCallback`? `useMemo` memoizes the *result* of an expensive calculation. `useCallback` memoizes a *function definition* so that its reference remains consistent across renders.

34. Explain `useRef`. It returns a mutable ref object whose `.current` property holds a value. It is used to access DOM elements directly or to store mutable values that should *not* cause re-renders when changed.

35. What is a Custom Hook? A JavaScript function whose name starts with "use" and that may call other Hooks. It is used to extract and reuse stateful logic across multiple components.

Routing & APIs

36. Explain React Router. React Router is the standard routing library for React. It keeps the UI in sync with the URL and enables navigation without full page reloads.

37. What is the `<Outlet />` component? Used in nested routing (React Router v6+), it tells the parent route component exactly where to render its child route components.

38. How do you implement a Protected Route? By creating a wrapper component that checks an authentication token (via Context or State). If valid, it renders the `<Outlet />` or children; if invalid, it returns a `<Navigate to="/login" />` component.

39. How do you fetch data in React? Usually within a `useEffect` hook (running on mount or when specific IDs change) using `fetch` or `axios`, storing the result in a local `useState` variable.

40. Why are libraries like React Query preferred over standard `useEffect`? They automatically handle caching, background refetching, dedpulication, loading states, error states, and pagination without requiring verbose boilerplate code.

Advanced & Performance

41. What is React.memo? A Higher-Order Component that prevents a functional component from re-rendering if its props have not changed (shallow comparison).

42. What is Code Splitting? Code splitting divides the final Javascript bundle into smaller chunks that can be loaded lazily on demand. This significantly reduces the initial load time of the application.

43. How do you implement Lazy Loading in React? Using `React.lazy()` for dynamic imports combined with the `<Suspense>` component to show a fallback UI while the chunk downloads.

44. What is a portal in React? React Portals provide a way to render children into a DOM node that exists outside the DOM hierarchy of the parent component. Widely used for Modals, Tooltips, and Dropdowns to escape `z-index` trapping.

45. What are Error Boundaries? Components that catch Javascript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of crashing the whole component tree. They currently must be Class components.

TypeScript in React

46. Why use TypeScript with React? It provides static typing, which catches bugs at compile time (like missing props or typos), improves IDE autocompletion, and makes component contracts safer and self-documenting.

47. What is the difference between `interface` and `type`? Both define shapes of objects. `interface` is restricted to object shapes and can be extended (`extends`). `type` can define aliases for primitives, unions, and tuples. In modern TS, they are mostly interchangeable for typing React props.

48. How do you type a standard functional component? You type the props object itself: `function Button(props: ButtonProps) {}`. Alternatively, `const Button: React.FC<ButtonProps> = (props) => {}`.

49. How do you type `useState` for a complex object? Provide the interface as a generic type argument: `const [user, setUser] = useState<User | null>(null)`.

50. How do you type an event handler? By importing React utility types. E.g., `(e: React.ChangeEvent<HTMLInputElement>)` for input changes, or `(e: React.FormEvent<HTMLFormElement>)` for form submissions.

Part 17: Coding Interview Problems

Real-world coding exercises commonly asked in frontend interviews. Solutions are written in React + TypeScript.

Exercise 1: Create a Counter Hook

Problem: Create a custom hook `useCounter` that takes an initial value and a step. It should return the current count, and functions to increment, decrement, and reset.

Solution:

```
import { useState } from 'react';

interface UseCounterOutput {
  count: number;
  increment: () => void;
  decrement: () => void;
  reset: () => void;
}

export function useCounter(initialValue: number = 0, step: number = 1):
UseCounterOutput {
  const [count, setCount] = useState<number>(initialValue);

  const increment = () => setCount((prev) => prev + step);
  const decrement = () => setCount((prev) => prev - step);
  const reset = () => setCount(initialValue);

  return { count, increment, decrement, reset };
}
```

Exercise 2: Debounced Search Input

Problem: Build an input field that fetches a list of users based on the search term. Ensure the API is not called on every single keystroke, but only when the user stops typing for 500ms.

Solution:

```
import React, { useState, useEffect } from 'react';

// Reusable hook
function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const handler = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(handler);
  }, [value, delay]);

  return debouncedValue;
}

// Component
export function UserSearch() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState<string[]>([]);
  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {
    if (debouncedQuery) {
      // Simulate API call
      console.log(`Fetching data for: ${debouncedQuery}`);
      setResults([`${debouncedQuery} result 1`, `${debouncedQuery} result 2`]);
    } else {
      setResults([]);
    }
  }, [debouncedQuery]);

  return (
    <div>
      <input
        type="text"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search..."
      />
      <ul>
        {results.map((r, i) => <li key={i}>{r}</li>)}
      </ul>
    </div>
```

```
  );
}
```

Exercise 3: Modal Component with Portal and Click Outside

Problem: Create a reusable Modal component that renders outside the main DOM tree, trapping focus, and closes when the user clicks the overlay mask or presses the Escape key.

Solution:

```
import React, { useEffect } from 'react';
import { createPortal } from 'react-dom';

interface ModalProps {
  isOpen: boolean;
  onClose: () => void;
  title: string;
  children: React.ReactNode;
}

export function Modal({ isOpen, onClose, title, children }: ModalProps) {
  // Handle escape key
  useEffect(() => {
    const handleEsc = (e: KeyboardEvent) => {
      if (e.key === 'Escape') onClose();
    };
    window.addEventListener('keydown', handleEsc);
    return () => window.removeEventListener('keydown', handleEsc);
  }, [onClose]);

  if (!isOpen) return null;

  const modalContent = (
    <div
      className="modal-overlay"
      onClick={onClose}
      style={{
        position: 'fixed', top: 0, left: 0, right: 0, bottom: 0,
        backgroundColor: 'rgba(0,0,0,0.5)',
        display: 'flex', justifyContent: 'center', alignItems: 'center'
      }}
    >
      <div
        className="modal-content"
```

```
        onClick={(e) => e.stopPropagation()} // Prevent bubbling to overlay
        style={{
          background: 'white', padding: '20px', borderRadius: '8px', minWidth:
'300px'
        }}
      >
        <div style={{ display: 'flex', justifyContent: 'space-between' }}>
          <h2>{title}</h2>
          <button onClick={onClose}>X</button>
        </div>
        <div className="modal-body">
          {children}
        </div>
      </div>
    </div>
  );

  // Render into document.body instead of deeply nested DOM
  return createPortal(modalContent, document.body);
}
```

Exercise 4: Simple Pagination

Problem: Create a component that accepts a large array of strings and displays them 5 at a time with "Next" and "Previous" buttons.

Solution:

```
import React, { useState } from 'react';

interface PaginationProps {
  data: string[];
  itemsPerPage?: number;
}

export function PaginatedList({ data, itemsPerPage = 5 }: PaginationProps) {
  const [currentPage, setCurrentPage] = useState(1);

  const totalPages = Math.ceil(data.length / itemsPerPage);

  // Calculate bounds
  const startIndex = (currentPage - 1) * itemsPerPage;
  const currentItems = data.slice(startIndex, startIndex + itemsPerPage);
```

```
  const handleNext = () => {
    setCurrentPage((prev) => Math.min(prev + 1, totalPages));
  };

  const handlePrev = () => {
    setCurrentPage((prev) => Math.max(prev - 1, 1));
  };

  return (
    <div>
      <ul>
        {currentItems.map((item, id) => (
          <li key={id}>{item}</li>
        ))}
      </ul>
      <div style={{ display: 'flex', gap: '10px', alignItems: 'center' }}>
        <button onClick={handlePrev} disabled={currentPage === 1}>
          Previous
        </button>
        <span>Page {currentPage} of {totalPages}</span>
        <button onClick={handleNext} disabled={currentPage === totalPages}>
          Next
        </button>
      </div>
    </div>
  );
}
```